



מכללת כנפי רוח, קרית נוער ירושלים

סמל מוסד 140129

ספר פרויקט לעבודת גמר

י"ד הנדסאי תוכנה (שאלון 714918)

בנושא:

SmallTalk פלטפורמה להעברת נתונים מקצה לקצה בצורה מאובטחת



מגיש: מיכאל אוליאל

ת"ז: 214407017

מנחה: אילן פרץ

תשפ"ה 2025

1	הצעת הפרויקט שאושרה ע"י משרד החינוך	1
2	מבוא	2
2.1	הרקע לפרויקט	2.1
2.2	תהליך המחקר	2.2
2.4	אתגרים מרכזיים במחקר	2.4
3	מטרות ויעדים	3
4	אתגרים בבניית המערכת	4
5	הגדרת מדדי הצלחה למערכת	5
6	רקע תיאורטי וספרות מקצועית	6
7	תיאור מצב קיים	7
8	ניתוח חלופות מערכתי	8
9	תיאור החלופה הנבחרת ונימוקים לבחירתה	9
10	אפיון המערכת המוצעת	10
10.1	ניתוח הדרישות מהמערכת	10.1
10.2	מודולי המערכת	10.2
10.3	אפיון פונקצונאלי וביצועים עקריים	10.3
10.4	אילוצי המערכת	10.4
11	תיאור תהליכי אבטחת מידע במערכת (רק למי שיש)	11
11.1	תיאור התקיפה / ההגנה	11.1
11.2	תיאור הצפנות	11.2
12	למידת מכונה (רק למי שבחר)	12
12.1	תהליך איסוף הנתונים	12.1
12.2	סוג הלמידה שנבחר	12.2
12.3	נוסחאות ורשתות למידה	12.3
13	תיכון המערכת	13
13.1	ארכיטקטורת המערכת (ע"פ מודל 3 השכבות)	13.1
13.2	תיכון מפורט של רכיבי המערכת	13.2
13.3	מודל שרת/לקוח	13.3
13.4	תיאור מסד הנתונים	13.4
13.5	תהליכים של מע' ההפעלה	13.5
13.6	תיאור פרוטוקולי תקשורת	13.6
13.7	ארכיטקטורת רשת (אם יש)	13.7
13.8	שירותים חיצוניים	13.8
14	ניתוח תרחישים וזרימת המידע	14
14.1	תרשימי תרחיש USE-CASE DIAGRAM	14.1
14.2	תרשימי רצף SEQUENCE DIAGRAM	14.2
15	תיאור מסכים וממשק משתמש	15
15.1	תרשים היררכיית המסכים	15.1
15.2	תיאור המסכים	15.2
16	תיאור התוכנה	16
16.1	סביבת עבודה	16.1
16.2	שפות תכנות	16.2
16.3	תיאור המודולים והמחלקות	16.3
16.4	מבנה נתונים בשימוש	16.4
16.5	קוד התוכנית לפעולות/פונקציות/אלגוריתמים עיקריות וחשובות	16.5
17	מדריך למשתמש	17
18	בדיקות והערכה	18
19	מסקנות	19
20	פיתוחים עתידיים	20
21	בבליוגרפיה	21

1 הצעת הפרויקט שאושרה ע"י משרד החינוך

פרטי המגיש

סמל מוסד: 140129

שם המכללה: כנפי רוח, קריית נוער ירושלים.

שם הסטודנט: מיכאל אוליאל ת"ז: 214407017

שם הפרויקט: SmallTalk פלטפורמה להעברת נתונים מקצה לקצה בצורה מאובטחת

פרטי הפרויקט

תיאור הפרויקט

תיאור הפרויקט SmallTalk היא אפליקציית Web לתקשורת מאובטחת. הפלטפורמה מספקת מענה לצורך הגובר בפרטיות ואבטחת מידע, באמצעות מנגנון הצפנת End-to-End האפליקציה מאפשרת שיחות פרטיות וקבוצתיות, שיתוף קבצים מוצפן בזמן אמת, ומבטיחה הגנה מלאה על הנתונים של המשתמשים ופרויקט זה שייך לתחום סייבר ואבטחת מידע וקריפטוגרפיה.

הגדרת הבעיה האלגוריתמית

ליבת הפרויקט מתמקדת בתחום אבטחת סייבר וקריפטוגרפיה, ובפרויקט זה אבנה יצירת מנגנון הצפנה ופענוח תוך שימוש באלגוריתמים מתקדמים כמו ECDH להחלפת מפתחות ו AES-256 להצפנת התוכן המיועד להבטיח את פרטיות המשתמשים ולשליחת הודעות מוצפנות.

2 מבוא

הרקע לפרויקט

2.1 הרקע לפרויקט

הפרויקט **SmallTalk** נועד לתת מענה לצורך הגובר בתקשורת דיגיטלית מאובטחת בעידן שבו פרטיות המשתמשים נמצאת תחת איום מתמיד מצד גורמים זדוניים. הפלטפורמה משתמשת במנגנון הצפנה מקצה-לקצה (**End-to-End**) כדי להבטיח סודיות מוחלטת של הנתונים, ומציעה שירותים כמו שיחות פרטיות וקבוצתיות ושיתוף קבצים מוצפן בזמן אמת.

2.2 תהליך המחקר

תהליך המחקר כלל מספר שלבים:

1. **סקירת ספרות מקצועית** בתחום הקריפטוגרפיה ואבטחת מידע, תוך התמקדות באלגוריתמים כמו **ECDH** ו-**AES-256**.
2. **ניתוח צרכים** של משתמשים באפליקציות תקשורת מאובטחות.
3. **בחינת חלופות טכנולוגיות**, כולל **RSA** ו-**DH**-לבחירת הפתרון האופטימלי.
4. **פיתוח ובדיקת פתרונות** באמצעות שימוש במנגנונים מתקדמים להבטחת פרטיות וביצועים גבוהים.

2.3 סקירת ספרות ומאמרים

הפרויקט מתבסס על מחקרים שמוכיחים את יעילות השילוב בין **ECDH** להחלפת מפתחות לבין **AES-256** להצפנה סימטרית:

- מחקר 1 הראה ששילוב זה משיג רמת אבטחה גבוהה תוך שמירה על יעילות ביצועית בסביבות מוגבלות משאבים כמו **IoT**.
- מחקר 2 הדגיש את היתרונות של מנגנונים קלים ויעילים שמאפשרים חיסכון בזמן ובמשאבים תוך שמירה על רמת אבטחה גבוהה.

2.4 אתגרים מרכזיים במחקר

• 2.4.1 התמודדות עם הבעיה

האתגרים המרכזיים היו:

- מניעת גישה לתוכן ההודעות מכל גורם חיצוני, כולל השרת עצמו.
- יצירת ערוץ תקשורת מאובטח להחלפת מפתחות בצורה דינמית ומוגנת.
- שילוב מנגנוני אבטחה מתקדמים עם מהירות תגובה גבוהה בזמן אמת.

• 2.4.2 הסיבות לבחירת הנושא

הנושא נבחר בשל הצורך הגובר בתקשורת דיגיטלית מאובטחת בעידן שבו פרטיות המידע נמצאת בסיכון מתמיד, לצד חשיבותו הרבה בתחום הסייבר והקריפטוגרפיה.

• 2.4.3 על איזה צורך הפרויקט עונה ?

הפרויקט עונה על הצורך בתקשורת מאובטחת המבטיחה סודיות מוחלטת של נתונים, תוך מתן אפשרות לשיתוף קבצים ושיחות קבוצתיות בצורה מוגנת ומוצפנת.

• 2.4.4 הצגת הפתרונות שנבחרו לבעיה

הפתרונות שנבחרו כוללים:

- **ECDH (Elliptic Curve Diffie-Hellman)** להחלפת מפתחות מאובטחת מעל ערוץ פתוח.
- **AES-256** להצפנה סימטרית ברמת אבטחה גבוהה במיוחד של תוכן ההודעות.
- **חתימות דיגיטליות**: לאימות זהות המשתמשים ולהבטחת שלמות ההודעות.

3 מטרות ויעדים לפיתוח המערכת

• מטרות עיקריות:

1. **סודיות ההודעות:** מניעת גישה לתוכן ההודעות מכל גורם חיצוני, כולל השרת עצמו.
 2. **החלפת מפתחות מאובטחת:** יצירת ערוץ תקשורת שבו המפתחות המשותפים נוצרים בצורה דינמית ומוגנת.
 3. **אימות זהות:** מנגנון המאמת את זהות השולח והמקבל, ללא תלות בצד שלישי.
- ביצועים וסנכרון בזמן אמת:** שילוב אבטחה מתקדמת עם מהירות תגובה גבוהה.

• יעדים מרכזיים:

א. ניהול משתמשים ואימות זהות:

1. הרשמה מאובטחת ויצירת מפתחות קריפטוגרפיים אישיים (פרטי וציבורי).
2. אימות דו-שלבי (FA2) ושמירה על אבטחת חשבון המשתמש.

ב. יצירת ערוץ תקשורת מאובטח (בעיה אלגוריתמית)

1. ECDH להחלפת מפתחות:

- כל משתמש יוצר זוג מפתחות (ציבורי ופרטי).
- מחושב מפתח משותף בטוח (Shared Secret) באמצעות המפתח הציבורי של הצד השני והמפתח הפרטי של המשתמש.

2. AES - 256 להצפנת הודעות:

- המפתח המשותף משמש להצפנה ופענוח של כל הודעה.
- הודעה מוצפנת נשלחת יחד עם IV (Initialization Vector) ייחודי.

ג. שליחת הודעות מוצפנות

- הודעות עוברות הצפנה סימטרית באמצעות AES-256.

ד. קבלת הודעות ופענוח

- המשתמש מקבל הודעה ומפענח אותה באמצעות המפתח המשותף.

ה. סנכרון בזמן אמת וניהול הודעות

- התראות והודעות מתעדכנות בזמן אמת באמצעות WebSockets.
- מנגנון לניהול היסטוריית הודעות ושמירת פרטיות המשתמש.

ו. מערכת גיבוי ושחזור מתקדמת

מערכת SmallTalk כוללת מערכת גיבוי ושחזור מתקדמת המהווה חלק מארכיטקטורת האבטחה הכוללת מערכת זו מבטיחה את שלמות הנתונים ואת המשכיות השירות תוך שמירה על עקרונות ההצפנה מקצה לקצה.

רכיבי המערכת העיקריים:

1. מנגנון גיבוי אוטומטי מוצפן מערכת הגיבוי האוטומטית מבצעת גיבויים תקופתיים של כל המידע הקריטי במערכת, כולל היסטוריית שיחות, קבצים משותפים והגדרות משתמש. תהליך הגיבוי משתמש באותם עקרונות הצפנה של המערכת הראשית:
 - הצפנת נתונים באמצעות AES-256 עם מפתחות ייחודיים
 - שימוש ב ECDH להחלפת מפתחות בתהליך השחזור
 - חתימות דיגיטליות לאימות שלמות הגיבויים
2. מערכת שחזור מאובטחת המערכת מספקת מנגנון שחזור מאובטח המאפשר למשתמשים לשחזר את המידע שלהם במקרה של אובדן מכשיר או תקלה. תהליך השחזור כולל:
 - אימות זהות מחמיר באמצעות מנגנון דו-שלבי
 - פענוח הדרגתי של הנתונים המגובים
 - בדיקות שלמות ואימות של המידע המשוחזר
3. מדיניות ניהול נתונים המערכת כוללת מנגנון חכם לניהול מדיניות שמירה והשמדה של נתונים:
 - הגדרת תקופות שמירה לסוגי מידע שונים
 - מחיקה מאובטחת של נתונים שפג תוקפם
 - התראות אוטומטיות על מצב הגיבויים

4 אתגרים בבניית המערכת

במהלך תכנון ובניית מערכת – **SmallTalk** פלטפורמה להעברת נתונים מאובטחת מקצה לקצה – זהו מספר אתגרים מרכזיים, טכנולוגיים ומהותיים, המחייבים פתרונות מתקדמים ומדויקת של רכיבי אבטחה, ביצועים וניהול. להלן האתגרים המרכזיים:

1. מידע מקצה לקצה (End-to-End Encryption)

המטרה המרכזית של המערכת היא להבטיח סודיות מלאה של המידע. האתגר נובע מהצורך לשלב מנגנוני הצפנה חזקים (AES-256) עם פרוטוקולי החלפת מפתחות – (ECDH) מבלי לאחסן את המפתחות על השרת. בנוסף, יש צורך למנוע גישה לא מורשית מצד גורמי ביניים כולל השרת עצמו, דבר המחייב הצפנה בצד הלקוח ופענוח רק אצל המקבל.

2. ניהול מאובטח של מפתחות קריפטוגרפיים

אחד האתגרים הקריטיים הוא כיצד לנהל ולשמור על מפתחות ההצפנה בצורה בטוחה. אם מפתח פרטי נחשף – כל ההודעות המוצפנות הופכות לפגיעות. לכן, יש צורך ביישום מנגנונים לשמירת מפתחות בזיכרון מאובטח, (secure storage) או לחלופין, חישוב דינמי של המפתח באמצעות ECDH בכל פתיחת שיחה.

3. אימות זהות המשתמשים

המערכת צריכה לאמת את זהות המשתמשים בצורה חד-משמעית, ללא שימוש בגורם שלישי מרכזי) כמו Google או Facebook לשם כך, יושם מנגנון של חתימות דיגיטליות (ECDSA) ואימות דו-שלבי, (2FA) המוודאים שההודעות נשלחות רק על ידי גורם מאומת ושלא נעשתה בהם כל מניפולציה בדרך.

4. תפקוד בזמן אמת וביצועים תחת עומס

מערכת התקשורת נדרשת לפעול בצורה מידית וללא השהיות משמעותיות, גם בתנאים של ריבוי משתמשים. השימוש ב WebSocket מאפשר תקשורת דו-כיוונית בזמן אמת, אך מצריך טיפול יעיל בניהול חיבורים פתוחים רבים במקביל, מניעת עומס יתר על השרת, ואופטימיזציה של תהליכי הצפנה/פענוח שלא יפגעו בזמני תגובה.

5. סנכרון הודעות והיסטוריה מוצפנת

שמירה על היסטוריית שיחות בצורה מוצפנת מציבה אתגר כפול: גם לשמור על פרטיות התוכן, וגם לאפשר למשתמש לשחזר הודעות באופן מאובטח. בנוסף, אם משתמש עובר בין מכשירים – המידע המוצפן צריך להיות משוחזר תוך שמירה על סודיות מלאה, כולל מנגנוני גיבוי מוצפן ובדיקת שלמות החומר.

6. שילוב בין ממשקים וטכנולוגיות מגוונות

המערכת מבוססת על ארכיטקטורת שלוש שכבות Vaadin, בצד הלקוח Spring Boot, בצד השרת, ו-MongoDB-לאחסון. השילוב בין טכנולוגיות אלו מחייב תיאום מדויק בין הממשק, הלוגיקה והנתונים – במיוחד בנושאים של הצפנה ופענוח בשני הקצוות. חוסר תיאום עשוי לגרום לשגיאות, תקלות באימות החתימות הדיגיטליות, או חוסר תאימות בין גרסאות.

7. שמירה על פרטיות גם במידע מטא-דאטה

מעבר להצפנה של תוכן ההודעות, ישנו אתגר נוסף: הגנה על מידע עקיף – כמו תאריך השליחה, זהות השולח/מקבל, ומידע על הקבצים. אתגר זה מצריך הצפנה של המטא-דאטה או אחסונו במבנה שאינו מאפשר קישור ישיר למידע רגיש.

5 הגדרת מדדי הצלחה למערכת

כדי להעריך את הצלחת מערכת **SmallTalk** נדרשת הגדרה ברורה של מדדים (KPIs – Key Performance Indicators) המתייחסים לאיכות הפיתוח, האבטחה, הביצועים וחויית המשתמש. מדדים אלו נבחנים הן ברמה הטכנית והן ברמה הפונקציונלית.

1. אבטחה ופרטיות

- **100% הצפנה מקצה לקצה – (E2EE)** כל הודעה, קובץ ונתון מועברים מוצפנים, ונפענחים רק בצד הלקוח.
- **עמידה במבחני חדירה בסיסיים – (Penetration Tests)** המערכת תעבור בדיקות חדירה פנימיות להבטחת עמידות בפני תקיפות נפוצות (כמו, Man-in-the-middle, XSS, SQL Injection).
- **אימות מוצלח של חתימות דיגיטליות ב-100% מהמקרים** – כל הודעה חתומה תאומת על ידי הצד המקבל ללא שגיאות.

2. ביצועים וזמן תגובה

- **זמן שליחת הודעה ממוצע נמוך מ-100 ms** – באמצעות WebSocket והצפנה יעילה.
- **קיבולת של לפחות 100 משתמשים בו-זמנית ללא פגיעה בביצועים** – מדד זה נבחן באמצעות סימולציות עומס.
- **זמן הצפנה/פענוח ממוצע לכל הודעה – 50ms <** כולל תהליך יצירת מפתח, הצפנה, שליחה ופענוח.

3. חויית משתמש

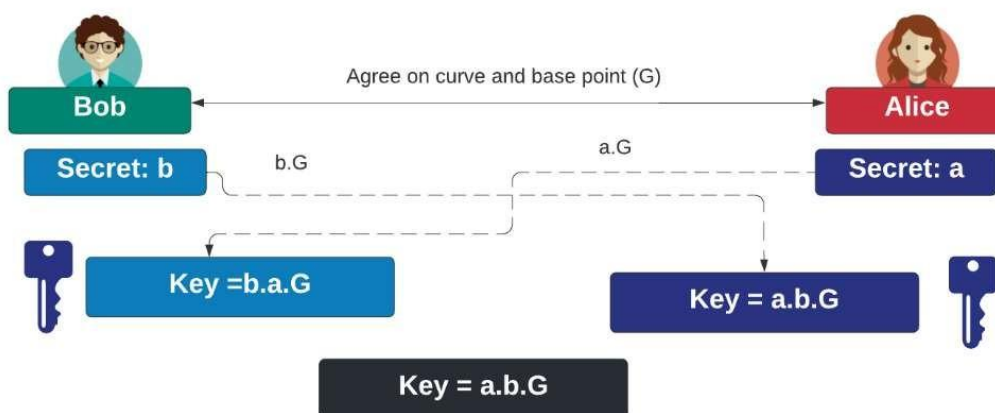
- **ממשק משתמש תקין ונגיש** – לא נרשמות שגיאות בטעינה, כל הכפתורים פעילים ונגישים.
- **שיעור הצלחה של פעולות בסיסיות (התחברות, שליחת הודעה, שיתוף קובץ) $\geq 98\%$** .
- **עיצוב ידידותי ואינטואיטיבי** – לפי משוב ממשתמשים בניסוי קבלה

6 רקע תיאורטי וספרות מקצועית

קריפטוגרפיה:

בסעיף זה אסביר על אלגוריתמים ECDH ו AES 256 וחתימה דיגיטלית

- ECDH (Elliptic Curve Diffie-Hellman) אלגוריתם להחלפת מפתחות מאובטחת המבוסס על עקומות אליפטיות.
- הפרוטוקול מאפשר לשני משתתפים שלא נפגשו מעולם ואינם חולקים ביניהם סוד משותף כלשהו מראש, להעביר אחד לשני מעל גבי ערוץ פתוח (שאינו מאובטח) סוד כלשהו כך שאיש מלבדם אינו יודע מהו. הסוד יכול להיות, בהקשר של פרוטוקול תקשורת מאובטח, מפתח הצפנה. פרוטוקול דיפי-הלמן מתמודד עם בעיית הפצת המפתחות על ידי פונקציה חד-כיוונית.



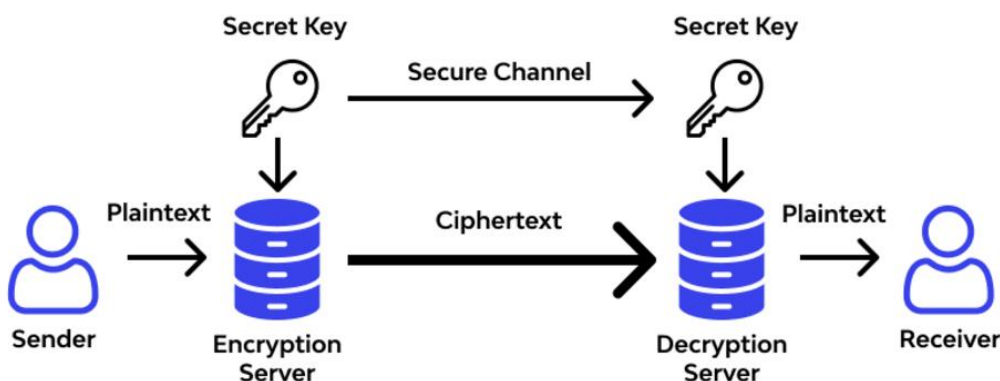
ניתן לראות באיור הנ"ל: ECDH הוא פרוטוקול לחילופין של מפתחות בין שני משתמשים,

המאפשר להם ליצור מפתח סודי משותף. הפרוטוקול עובד בשלושה שלבים עיקריים:

1. בוב ואליס מסכימים על עקום אליפטי ונקודת בסיס משותפים.
2. כל אחד מהם בוחר מספר סודי משלו, ומחשב מפתח ציבורי על בסיס זה.
3. הם חולקים את המפתחות הציבוריים, ואז כל אחד יכול לחשב את המפתח הסודי המשותף באמצעות המפתח הציבורי של הצד השני והמפתח הסודי שלו התהליך מאפשר לבוב ואליס לשתף מפתח סודי מבלי שתוקף פוטנציאלי יוכל לחשב אותו, בזכות העובדה שהבעיה המתמטית שבבסיס זה היא מאוד קשה לפתרון.

- **AES-256 (Advanced Encryption Standard)** אלגוריתם סימטרי להצפנת הודעות, המספק רמת אבטחה גבוהה במיוחד הצפנת AES-256 היא דרך לשמור על הודעות סודיות או מידע בטוח מאנשים שלא אמורים להיות מסוגלים לראות אותם. הצפנת AES-256 היא כמו שיש לך מנעול סופר חזק על הקופסה שלך שניתן לפתוח רק על ידי מפתח מאוד ספציפי. המנעול כל כך חזק שיהיה קשה מאוד למישהו לשבור אותו ולפתוח את הקופסה ללא המפתח הנכון.

AES Algorithm Working



ניתן לראות באיור את התהליך של האלגוריתם ואיך הוא עובד :

- AES הוא אלגוריתם הצפנה סימטרי, שמשמעותו שהשולח והמקבל משתמשים באותו מפתח סודי
1. השולח מזין את הטקסט הפשוט שהוא רוצה להעביר לשרת ההצפנה.
 2. השרת משתמש במפתח הסודי כדי להצפין את הטקסט, ויוצר טקסט מוצפן.
 3. הטקסט המוצפן נשלח דרך ערוץ מאובטח למקבל.
 4. המקבל משתמש באותו מפתח סודי כדי לפענח את הטקסט המוצפן בחזרה לטקסט המקורי.
- העיקרון הוא שהשולח והמקבל משתפים רק את המפתח הסודי ביניהם, מה שמאפשר להם להעביר מידע בצורה מאובטחת AES.

- **חתימות דיגיטליות**: מנגנון המבטיח את שלמות ההודעות ואותנטיות השולח חתימה דיגיטלית היא שיטה קריפטוגרפית, שמטרתה לאמת את המקוריות והשלמות של הודעה או מסמך דיגיטלי.
- חתימה דיגיטלית תקפה מעידה בסבירות גבוהה שההודעה או המסמך אינם משוברים או מזויפים, כלומר – שהמסמך או ההודעה נוצרו או נשלחו על ידי גורם

ידוע ושהם לא שונו על ידי צד שלישי לאחר חתימתם, או שובשו בדרך אחרת. כמו כן חותם המסמך לא יוכל להתכחש לכך שהוא יצר אותו.

ספרות מקצועית

- במאמר [1] יש מחקר המתאר יישום של ECDH ו AES בסביבות מוגבלות משאבים כמו . IoT . המחקר מראה ששילוב של ECDH ליצירת מפתחות ואימות עם AES להצפנה סימטרית משיג רמת אבטחה גבוהה תוך שמירה על יעילות ביצועית. בניסויים שיעור ההצלחה עמד על 99.9% וזמני ההצפנה הממוצעים היו מהירים ועמדו על 85 ms בניסיון הראשון ו 65 ms בניסיון השני. המסקנות מהמחקר תומכות בשילוב דומה בפרויקט SmallTalk כדי לספק הצפנה חזקה וביצועים מעולים.
- במאמר [2] המציג שילוב של ECDH להחלפת מפתחות ואימות עם AES להצפנה. המחקר מותאם למגבלות המשאבים של מערכות IoT ומדגיש את היתרונות של שימוש במנגנונים קלים ויעילים המאפשרים חיסכון בזמן ובמשאבים, תוך שמירה על רמת אבטחה גבוהה. ממצאי המחקר מחזקים את הבחירה בשיטות אלו בפרויקט SmallTalk כדי להתמודד עם אתגרי הביצועים והאבטחה.

7 תיאור מצב קיים וניתוח חלופות למערכת

- אפשרויות קיימות:

1. Signal

- מבוססת על פרוטוקול Signal המשלב ECDH ו AES
- דורשת מספר טלפון לזיהוי משתמשים, מוגבלת במנגנוני ניהול חשבונות.

2. WhatsApp

- משתמשת ב Signal Protocol אך כוללת איסוף נתונים לצרכים מסחריים.

3. Telegram

- תומכת בהצפנה מקצה לקצה רק בצ'אטים פרטיים, עם דגש פחות על פרטיות מוחלטת.

8 ותיאור החלופה הנבחרת ונימוקים לבחירתה

- בחנתי מספר גישות חלופיות להחלפת מפתחות ולהצפנת נתונים על מנת להבטיח את רמת האבטחה הגבוהה ביותר לצד ביצועים מיטביים. אלו האפשרויות שנבחנו:
 - א. אלגוריתמים להחלפת מפתחות
 - RSA (Rivest-Shamir-Adleman)
 - שיטה מבוססת קריפטוגרפיה אסימטרית, המתבססת על הקושי שבפירוק לגורמים ראשוניים. RSA הוא פרוטוקול בוגר ומוכר, אך הוא פחות יעיל מבחינת ביצועים, בעיקר עם הגדלת אורך המפתח לצורך הבטחת רמת אבטחה מודרנית.
 - DH (Diffie-Hellman)
 - הגרסה המסורתית של פרוטוקול דיפי-הלמן. אמנם מספק אבטחה נאותה, אך אינו יעיל כמו ECDH, במיוחד בסביבות מוגבלות משאבים או מערכות עם דרישות ביצועים גבוהות.
 - ב. אלגוריתמים להצפנה סימטרית
 - ChaCha20
 - אלגוריתם הצפנה קל ומהיר יותר מ-AES. מתאים לסביבות עם מגבלות משאבים, אך פחות נפוץ בשימוש במערכות מבוססות תקנים תעשייתיים
 - DES (Data Encryption Standard)
 - אלגוריתם הצפנה סימטרי ותיק המשתמש באורך מפתח של 56 ביטים וצופן בלוקים בגודל 64 ביטים. DES היה תקן אבטחה פופולרי בשנות ה-70 וה-80, אך כיום נחשב למיושן ובלתי בטוח בשל אורך המפתח הקצר, שמאפשר לבצע התקפות כוח גס (brute force) בזמן סביר. בעוד שהוא פשוט וקל ליישום, הוא לא עומד בדרישות האבטחה המודרניות.
 - Triple DES
 - גרסה מורחבת של DES, המספקת רמת אבטחה גבוהה יותר. עם זאת, נחשב לפחות יעיל ופחות בטוח לעומת AES-256, במיוחד אל מול התקפות מודרניות.
 - ג. חתימות דיגיטליות

• RSA Digital Signature

שיטה מבוססת RSA ליצירת חתימות דיגיטליות. מספקת אבטחה טובה, אך פחות יעילה ויותר מסורבלת לעומת חתימות מבוססות ECDSA, המותאמות לקריפטוגרפיה על עקומות אליפטיות.

להלן השוואה בין החלופות על פי מאפיינים והתאמה לפרויקט זה

פרוטוקול/אלגוריתם	רמת אבטחה	ביצועים	התאמה לסביבות מוגבלות משאבים	התאמה לפרויקט SmallTalk
ECDH	גבוהה	גבוהה	טובה מאוד	מתאים מאוד
RSA	גבוהה	נמוכה	מוגבלת	מתאים פחות
DH (מסורתי)	בינונית	נמוכה	מוגבלת	מתאים פחות
AES-256	גבוהה	גבוהה	טובה	מתאים מאוד
ChaCha20	גבוהה	גבוהה מאוד	מצוינת	אופציונלי
Triple DES	בינונית	נמוכה	נמוכה	מתאים פחות

האלגוריתמים שנבחרו למימוש בפרויקט זה

הבחירה באלגוריתמים ECDH ו-AES-256 נובעת מהשילוב המיטבי שהם מספקים בין אבטחה ביצועים ויכולת התאמה לצרכי הפרויקט:

• ECDH

נבחר בזכות יעילותו הגבוהה בסביבות מודרניות, תוך שמירה על רמת אבטחה גבוהה. פרוטוקול זה מתאים במיוחד לסביבות מוגבלות משאבים כמו מערכות מבוזרות ואפליקציות בזמן אמת.

• AES-256

השימוש ב-AES-256 להצפנה סימטרית נבחר בשל השילוב האידיאלי בין פשטות יישום, תמיכה רחבה בתעשייה, ורמת אבטחה שאינה ניתנת לשבירה באמצעים ידועים.

בעוד DH ו RSA מהווים פרוטוקולים בוגרים ומוכרים, הם פחות יעילים מ ECDH בסביבות הדורשות ביצועים גבוהים. באותו אופן AES-256 נמצא מתאים במיוחד בזכות פשטות יישומו והאמינות הגבוהה שהוא מספק לאורך שנים

9 איפיון המערכת המוצעת

9.1 ניתוח הדרישות מהמערכת

המערכת SmallTalk היא פלטפורמה להעברת נתונים מאובטחת מקצה לקצה, ונדרשת לעמוד בדרישות רבות ברמת אבטחה, ביצועים, שימושיות ויכולת הרחבה.

דרישות פונקציונליות:

- הרשמה מאובטחת ואימות דו-שלבי (2FA).
- יצירת שיחות פרטיות וקבוצתיות.
- העברת הודעות מוצפנות בזמן אמת.
- שיתוף קבצים מאובטח ומוצפן.
- אימות זהות באמצעות חתימות דיגיטליות.
- מערכת גיבוי ושחזור מוצפנת ומאובטחת.
- תמיכה בתקשורת דו-כיוונית בזמן אמת (WebSocket).

דרישות לא-פונקציונליות:

- **ריבוי משתמשים:** תמיכה בעומסים של מאות משתמשים בו-זמנית.
- **שמירת נתונים:** כל הנתונים מאוחסנים בצורה מוצפנת במסד NoSQL (MongoDB).
- **יעילות ומהירות:** הצפנת והעברת הודעות תוך פחות מ-2 שניות.
- **אבטחת מידע גבוהה:** שימוש ב ECDH, AES-256, ECDSA, ו-SSL/TLS.
- **זמינות גבוהה:** זמינות של 99.9% לפחות.
- **אינטגרציה עם טכנולוגיות מודרניות:** Vaadin, Spring Boot, MongoDB.

9.2 מודולי המערכת

1. מודול ניהול משתמשים:

- הרשמה, התחברות ואימות דו-שלבי.
- ניהול מפתחות הצפנה לכל משתמש.

2. מודול שיחות:

- יצירת וניהול שיחות פרטיות או קבוצתיות.
- שליחת וקבלת הודעות מוצפנות.

3. מודול הצפנה/פענוח:

- החלפת מפתחות באמצעות ECDH.
- הצפנה סימטרית עם AES-256.
- אימות וחתימה דיגיטלית עם ECDSA.

4. מודול שיתוף קבצים:

- העלאת והורדת קבצים בצורה מוצפנת.
- שימוש במפתחות ייחודיים לכל קובץ.

5. מודול סנכרון בזמן אמת:

- תקשורת באמצעות WebSocket.
- התראות והודעות מיידיות.

6. מודול גיבוי ושחזור:

- גיבוי אוטומטי של הודעות וקבצים.
- שחזור מאובטח לפי אימות זהות.

7. ממשק משתמש:

- בנוי עם Vaadin, כולל עיצוב רספונסיבי ונגיש.
- ממשק ניהול הודעות, קבצים וחשבונות משתמש.

9.3 אפיון פונקציונלי וביצועים עיקריים

- זמן ממוצע לשליחת הודעה מוצפנת: פחות מ-2 שניות.
- שחזור גיבוי מלא: תוך פחות מ-10 שניות לאחר אימות.
- התממשקות בזמן אמת עם שרת WebSocket.
- הצפנה ופענוח של קבצים בגודל של עד 16 MB בזמן ממוצע של עד 5 שניות.
- חתימה דיגיטלית ואימות של כל הודעה תוך פחות מ-1 שנייה.

9.4 אילוצי המערכת

- אבטחת מידע: כל המידע חייב להיות מוצפן לאורך כל זמן התעבורה והאחסון.
- שמירה על פרטיות המשתמש: אף צד שלישי (כולל השרת) לא יכול לגשת לתוכן.
- ביצועים בזמן אמת: תמיכה בשיחות והודעות ללא שיהוי משמעותי.
- תמיכה בפלטפורמות שונות: עבודה בדפדפנים מודרניים (Cross-Browser).
- מוגבלות משאבים: נדרש לייעל הצפנה גם בסביבות עם משאבים מוגבלים (למשל במובייל).
- הסתמכות על תקני אבטחה מוכרים בלבד: אין לפתח אלגוריתמים פרטיים אלא להשתמש

בתקנים כמו AES, ECDH, ECDSA.

מודול ניהול משתמשים

מאפיין	תיאור
שם המודול	ניהול משתמשים ואימות זהות
תיאור	אחראי לרישום, כניסה, יצירת מפתחות הצפנה ואימות דו-שלבי.
פונקציות עיקריות	- יצירת חשבון משתמש - כניסה עם אימות דו-שלבי (2FA) - ניהול מפתחות קריפטוגרפיים (ציבורי ופרטי) - ניהול פרופיל משתמש
טכנולוגיות רלוונטיות	Spring Boot, Vaadin, MongoDB, JWT, 2FA API, Bcrypt

מודול שיחות

מאפיין	תיאור
שם המודול	ניהול שיחות
תיאור	יוצר ומנהל שיחות פרטיות וקבוצתיות בין משתמשים.
פונקציות עיקריות	- יצירת שיחה (פרטית/קבוצתית) - הוספה והסרה של משתתפים - אחסון מזהי שיחות ומטא-נתונים
טכנולוגיות רלוונטיות	MongoDB, REST API, JSON, UUID Generator

מודול הצפנה/פענוח

מאפיין	תיאור
שם המודול	הצפנה ופענוח מידע
תיאור	אחראי להצפנת ופענוח הודעות וקבצים לפי מפתח משותף מוצפן.
פונקציות עיקריות	- החלפת מפתחות באמצעות ECDH - הצפנת תוכן עם AES-256 - יצירת IV ייחודי - חתימה דיגיטלית באמצעות ECDSA
טכנולוגיות רלוונטיות	Java Cryptography API, BouncyCastle, AES, ECDH, ECDSA, SHA-256

 מודול שיתוף קבצים

מאפיין		תיאור
שם המודול	שיתוף קבצים מאובטח	
תיאור	מאפשר למשתמשים להעביר קבצים מוצפנים בצורה בטוחה.	
פונקציות עיקריות	- הצפנת קובץ לפי מפתח "חודי" - אחסון כתובת מוצפנת לקובץ - פענוח קבצים בצד הלקוח - הצגת סוג קובץ וסטטוס שליחה	
טכנולוגיות רלוונטיות	AES-256, Multipart Upload, MongoDB GridFS, File IO, Base64	

 מודול סנכרון בזמן אמת

מאפיין		תיאור
שם המודול	תקשורת בזמן אמת	
תיאור	מאפשר קבלת ושליחת הודעות בזמן אמת בין משתמשים.	
פונקציות עיקריות	- פתיחת חיבור WebSocket - שליחת הודעות מיידיות - קבלת התראות ועדכונים - שמירה על חיבור יציב	
טכנולוגיות רלוונטיות	WebSocket, STOMP, Spring Messaging, JSON	

10

תיאור התקיפה / ההגנה**תיאור התקיפה / ההגנה****1. התקפת Man-in-the-Middle (MitM)**

- **תיאור התקיפה:** תוקף מנסה להתמקם בין שני משתמשים כדי ליירט ולשנות את התקשורת ביניהם.
- **מנגנון הגנה:** מערכת SmallTalk מיישמת פרוטוקול ECDH להחלפת מפתחות שמונע מהתוקף לחשב את המפתח המשותף גם אם הוא מצליח ליירט את המפתחות הציבוריים. בנוסף, מימוש חתימות דיגיטליות ECDSA מאפשר לזהות שינויים בהודעות.

2. התקפות כוח גס על מפתחות הצפנה

- **תיאור התקיפה:** תוקף מנסה לפצח את ההצפנה באמצעות בדיקה שיטתית של כל המפתחות האפשריים.
- **מנגנון הגנה:** מערכת SmallTalk משתמשת בהצפנת AES-256 עם מפתחות באורך 256 סיביות, מה שהופך התקפת כוח גס לבלתי אפשרית מבחינה חישובית עם הטכנולוגיה הקיימת כיום.

3. התקפת שחזור מפתחות

- **תיאור התקיפה:** ניסיון לשחזר מפתחות פרטיים מתוך מידע ציבורי.
- **מנגנון הגנה:** המערכת מתבססת על הקושי המתמטי של בעיית הלוגריתם הדיסקרטי בעקומות אליפטיות, שהופכת את שחזור המפתח הפרטי מהמפתח הציבורי לבלתי אפשרי מבחינה חישובית.

4. התקפות Replay

- **תיאור התקיפה:** תוקף מקליט הודעות מוצפנות ושולח אותן מחדש בשלב מאוחר יותר.
- **מנגנון הגנה:** המערכת משתמשת בוקטורי אתחול (IV) ייחודיים לכל הודעה, מה שמבטיח שגם אם תוכן ההודעה זהה, ההצפנה תהיה שונה בכל פעם.

5. התקפות על שרת האחסון

- **תיאור התקיפה:** ניסיון לגישה למידע רגיש המאוחסן בשרתי המערכת.
- **מנגנון הגנה:** תודות להצפנת End-to-End גם אם תוקף משיג גישה לשרתי המערכת, הוא לא יוכל לפענח את תוכן ההודעות ללא המפתחות הפרטיים של המשתמשים.

6. התקפות התחזות (Impersonation)

- **תיאור התקיפה:** תוקף מנסה להתחזות למשתמש לגיטימי כדי לתקשר עם משתמשים אחרים.

- **מנגנון הגנה:** המערכת מיישמת אימות דו-שלבי (2FA) ומערכת חתימות דיגיטליות המאפשרות למקבל לאמת את זהות השולח באמצעות המפתח הציבורי שלו.

מנגנוני אבטחה והגנה נוספים:

1. פרוטוקולי TLS/SSL

- מבטיחים תקשורת מאובטחת בין הלקוח לשרת, כשכבת הגנה נוספת מעבר להצפנת End-to-End.

2. מערכת גיבוי ושחזור מאובטחת

- גיבויים מוצפנים באמצעות AES-256
- שימוש בחתימות דיגיטליות לאימות שלמות הגיבויים
- מחיקה מאובטחת של נתונים שפג תוקפם

3. ניהול מפתחות מאובטח

- אחסון מאובטח של מפתחות פרטיים
- מנגנוני החלפת מפתחות מבוססי ECDH
- תהליכי גזירת מפתחות (Key Derivation) להפקת מפתחות הצפנה סופיים

4. אימות זהות רב-שכבתי

- אימות דו-שלבי (2FA)
- חתימות דיגיטליות לאימות זהות השולח
- מנגנוני בדיקת שלמות ההודעות

10.2 תיאור הצפנות

מנגנוני הצפנה מרכזיים:

1.

- **ECDH (Elliptic Curve Diffie-Hellman)** אלגוריתם להחלפת מפתחות מאובטחת המבוסס על עקומות אליפטיות.
- הפרוטוקול מאפשר לשני משתתפים שלא נפגשו מעולם ואינם חולקים ביניהם סוד משותף כלשהו מראש, להעביר אחד לשני מעל גבי ערוץ פתוח (שאינו מאובטח) סוד כלשהו כך שאיש מלבדם אינו יודע מהו. הסוד יכול להיות, בהקשר של פרוטוקול תקשורת מאובטח, מפתח הצפנה. פרוטוקול דיפיהלמן מתמודד עם בעיית הפצת המפתחות על ידי פונקציה חד-כיוונית.

○ תהליך :

- כל משתמש יוצר זוג מפתחות : מפתח פרטי (d) ומפתח ציבורי ($Q = d \times G$), כאשר G היא נקודת בסיס על העקומה האליפטית
- המשתמשים מחליפים ביניהם רק את המפתחות הציבוריים
- כל משתמש מחשב את המפתח המשותף ($K = d \times Q_{\text{other}}$): מפתח פרטי שלו כפול המפתח הציבורי של הצד השני)

○ **יתרונות:** מספק אבטחה גבוהה עם מפתחות קצרים יותר בהשוואה ל RSA או DH

רגיל, מה שמספר את הביצועים תוך שמירה על רמת אבטחה גבוהה.

2. (אלגוריתם סימטרי להצפנת הודעות, המספק רמת אבטחה גבוהה במיוחד הצפנת AES-256 היא דרך לשמור על הודעות סודיות או מידע בטוח מאנשים שלא אמורים להיות מסוגלים לראות אותם. הצפנת AES-256 היא כמו שיש לך מנעול סופר חזק על הקופסה שלך שניתן לפתוח רק על ידי מפתח מאוד ספציפי. המנעול כל כך חזק שיהיה קשה מאוד למישהו לשבור אותו ולפתוח את הקופסה ללא המפתח הנכון.

יצירת מפתחות (Key Generation)

כל משתמש בגרסה של אלגוריתם הצפנה מבוסס על קריפטוגרפיה של עקומות אליפטיות (Elliptic Curve Cryptography, ECC) יוצר זוג מפתחות :

• מפתח פרטי (Private Key)

המפתח הפרטי הוא למעשה "המפתח הסודי" של המשתמש. הוא מיוצר כמספר אקראי בעל אורך קבוע (לדוגמה 256 סיביות), והוא צריך להישמר בסודיות מוחלטת. דמיינו את המפתח הפרטי כמו מפתח של כספת אישית - אף אחד לא צריך לדעת עליו חוץ מבעל הכספת עצמו.

• מפתח ציבורי (Public Key)

המפתח הציבורי נוצר על ידי ביצוע פעולה מתמטית על המפתח הפרטי באמצעות העקומה האליפטית. הנוסחה המתמטית $d \times G$ יוצרת את המפתח הציבורי Q , כאשר :

- d הוא המפתח הפרטי שמקבל מספר אקראי גדול ביותר שנבחר בטווח מסוים.
- G היא נקודת הבסיס על העקומה האליפטית נקודת הבסיס והיא נקודה קבועה ומוגדרת מראש על העקומה האליפטית. היא משמשת כ"נקודת התחלה" לחישוב המפתח הציבורי.
- Q הוא המפתח הציבורי והוא למעשה נקודה גיאומטרית על העקומה האליפטית, המתקבלת על ידי פעולת הכפל.

דוגמה להמחשה

נניח שיש לנו מפתח פרטי $d = 6$, ונקודת בסיס G על העקומה.

נבצע את החישוב: $x \cdot G = Q$ שהוא המפתח הציבורי.

החלפת מפתחות (Key Exchange)

בשלב זה, שני משתמשים A ו B רוצים להסכים על מפתח משותף שמאפשר הצפנה ופענוח של הודעות ביניהם, מבלי שהם ישתפו במפתח הפרטי שלהם.

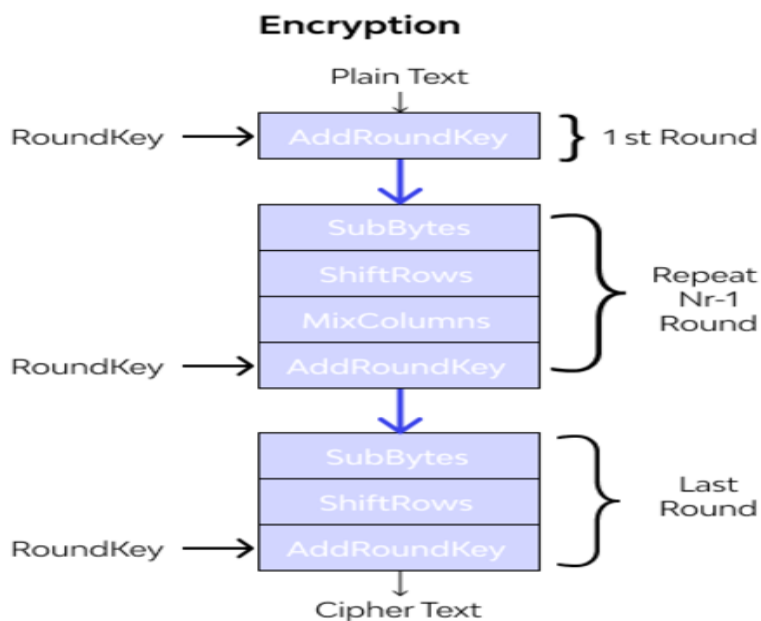
- כל אחד מהמשתמשים שולח את המפתח הציבורי שלו לשני הצדדים.
- משתמש A מחשב את המפתח המשותף K באמצעות המפתח הפרטי שלו A^d
- והמפתח הציבורי של משתמש B B^Q

$$B^Q \times A^d = K$$

משתמש B מחשב את המפתח המשותף K בעזרת המפתח הפרטי B^Q שלו והמפתח הציבורי של

משתמש A A^d .

הצפנת הודעות (Encryption)



פה בתמונה ניתן לראות איך ההצפנה פועלת

לאחר שהשניים הסכימו על המפתח המשותף K הם יכולים להשתמש בו כדי להצפין את ההודעות:

- המפתח המשותף K עובר בדרך כלל תהליך של גזירת מפתח (Key Derivation) על מנת להפיק מפתח חזק יותר וייחודי להצפנה. לדוגמה, ניתן להשתמש בפונקציות Hash כמו SHA-256 על מנת ליצור את המפתח הסופי $Final^K$.

$$SHA-256(K) = final^K$$

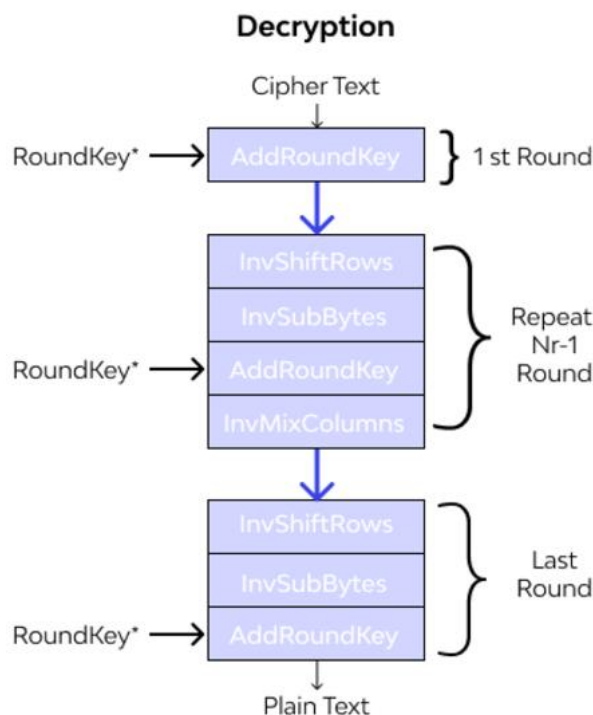
הודעה M מוצפנת באמצעות אלגוריתם AES-256 אלגוריתם הצפנה סימטרי:

$$\text{Encrypt}_{\text{final}K}(M) = C$$

בכדי למנוע התקפות הצפנה חוזרת, מצורף לנתונים וקטור אתחול (IV) לכל הודעה, מה שהופך את הצפנת ההודעה לאי-חזרתית.

וקטור אתחול (Initialization vector - IV) הוא מחרוזת סיביות ייחודית כלשהי, באורך מוגדר המשמשת להתחלת תהליך ההצפנה בצופן זרם או בצופן בלוקים כאשר עושים שימוש במצב הצפנה מסוג שרשור (Chaining).

פענוח הודעות (Decryption):



בתמונה פה ניתן לראות את הפענוח

- לאחר קבלת ההודעה המוצפנת, המקבל יכול לפענח אותה כך:

הוא מחשב את המפתח המשותף K בעזרת המפתח הפרטי שלו $receiver^d$

והמפתח הציבורי של השולח $sender^Q$

$$sender^Q \times receiver^d = K$$

לאחר מכן, הוא גוזר את המפתח הסופי $Final^K$ ומשתמש בו לפענוח ההודעה:

$$\text{Decrypt}_{\text{final}K}(C) = M$$

ההודעה המפוענחת M היא ההודעה המקורית שנשלחה על ידי השולח.

3. ECDSA (Elliptic Curve Digital Signature Algorithm) לחתימות דיגיטליות

- **תיאור:** אלגוריתם לחתימה דיגיטלית המבוסס על עקומות אליפטיות, המאפשר לאמת את מקור ושלמות ההודעה.
- **תהליך:**
 - השולח חותם על ההודעה באמצעות המפתח הפרטי שלו
 - החתימה הדיגיטלית נשלחת יחד עם ההודעה המוצפנת
 - המקבל מאמת את החתימה באמצעות המפתח הציבורי של השולח
- **יתרונות:** מבטיח את אותנטיות ההודעה, מונע התכחשות (non-repudiation), ומאפשר לזהות שינויים בהודעה במהלך העברתה.

תהליך הצפנה מלא:

1. **יצירת מפתחות:**
 - כל משתמש מייצר זוג מפתחות ECDH (פרטי וציבורי)
 - המפתח הפרטי נשמר במכשיר המשתמש בלבד, המפתח הציבורי משותף עם השרת
2. **החלפת מפתחות:**
 - כאשר משתמש A רוצה לתקשר עם משתמש B, המערכת מבצעת החלפת מפתחות ECDH
 - מחושב מפתח משותף ייחודי K לשני המשתמשים
3. **הצפנת הודעות:**
 - המפתח המשותף K עובר גזירת מפתח ליצירת מפתח הצפנה סופי K_Final
 - לכל הודעה נוצר IV ייחודי
 - ההודעה מוצפנת באמצעות AES-256 עם המפתח הסופי והוקטור
 - נוצרת חתימה דיגיטלית על ההודעה המוצפנת באמצעות ECDSA
 - ההודעה המוצפנת, ה-IV, והחתימה הדיגיטלית נשלחים לשרת
4. **פענוח הודעות:**
 - המקבל מאמת את החתימה הדיגיטלית באמצעות המפתח הציבורי של השולח

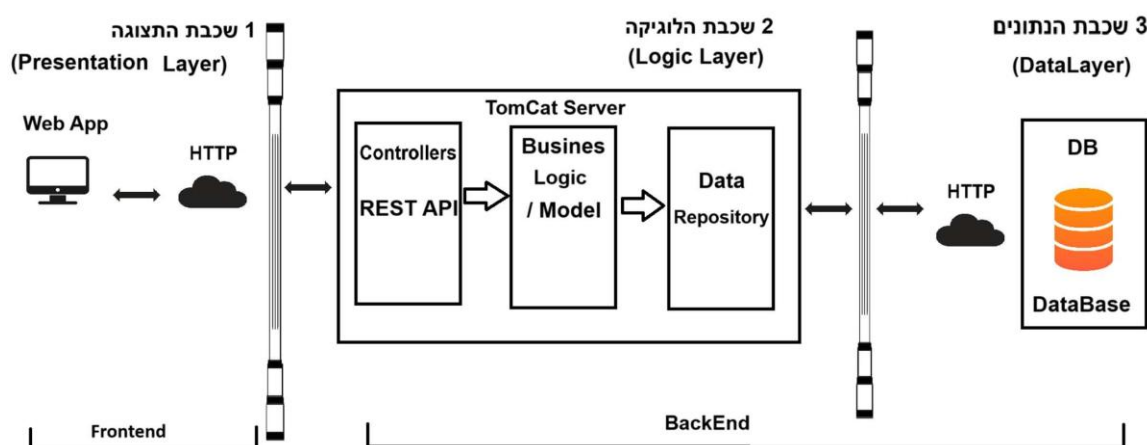
○ אם החתימה תקינה, המקבל משתמש במפתח המשותף K_Final שלו לפענוח ההודעה

באמצעות AES-256

תהליכי האבטחה המשולבים במערכת SmallTalk מבטיחים פרטיות מלאה ואבטחת מידע ברמה גבוהה, כאשר גם שרתי המערכת אינם יכולים לקרוא את תוכן ההודעות המוצפנות - מה שמבטיח הצפנת-End to-End אמיתית.

11 תיוכונ המערכת

ולהלן תרשים שלושת השכבות



11.1 ארכיטקטורת המערכת (ע"פ מודל 3 השכבות)

רכיטקטורת שלוש השכבות (Three-Tier Architecture) היא מודל נפוץ לפיתוח מערכות תוכנה, המפריד את המערכת לשלושה חלקים עיקריים: שכבת התצוגה, שכבת הלוגיקה ושכבת הנתונים. לכל שכבה בארכיטקטורה זו יש תפקיד ברור, והיא מתמקדת בהיבט אחר של המערכת. חלוקה זו נועדה לשפר את תחזוקת המערכת, לאפשר גמישות בשדרוגים ולייעל את העבודה בצוותי פיתוח.

11.2 תיוכונ מפורט של רכיבי המערכת

הסבר על התרשים שלושת השכבות

1. שכבת ממשק המשתמש (Presentation Layer)

- אחראית על אספקת ממשק משתמש מאובטח ונוח למשתמשים.
- ככל הנראה נבנית באמצעות טכנולוגיה מבוססת אינטרנט, כמו פלטפורמת Vaadin המאפשרת בניית ממשקים עשירים ותגובתיים בשפת Java.
- הממשק מתקשר עם שכבת הלוגיקה העסקית בשני אופנים:
 - דרך ממשק REST API לביצוע פעולות כמו התחברות, רישום וניהול פרופיל.
 - דרך חיבור WebSocket לצורך תקשורת בזמן אמת, כמו הודעות מיידיות.

- כדי להבטיח הצפנה מקצה לקצה (E2EE) של נתוני המשתמשים, הממשק מיישם הצפנה באמצעות שילוב של ECDH להחלפת מפתחות ו AES-256 להצפנת ההודעות.

2. שכבת הלוגיקה (Logic Layer):

- שרת TomCat אחראי על הלוגיקה העסקית המרכזית ועל ניהול הנתונים.
- חושפת מערך של ממשקי REST API שהממשק יכול לצרוך.
- מחולקת לשלושה רכיבים עיקריים:
 - בקרים (Controllers) טיפול בבקשות נכנסות, אימות קלט, וזימון הלוגיקה העסקית המתאימה.
 - שירותים (Services) מכילים את הלוגיקה העסקית העיקרית, כמו ניהול משתמשים, הודעות, שיתוף קבצים, והצפנה/פענוח.
 - מאגרים (Repositories) מספקים הפשטה לאחסון ואחזור הנתונים, כנראה באמצעות מסד נתונים NoSQL כמו MongoDB.
 - משלבת מנגנוני אבטחה כמו אימות, הרשאות, והצפנה, להגנה על האפליקציה ונתוני המשתמשים.
 - ככל הנראה מיושמת באמצעות פריימוורק Java כמו Spring Boot..

3. שכבת הנתונים (Data Layer)

- אחראית על אחסון ואחזור של נתוני המשתמשים, היסטוריית ההודעות, ומטא-נתוני הקבצים.
 - מחוברת לשכבת הלוגיקה העסקית ומספקת שירותי אחסון ואחזור נתונים.
 - בהתאם לשימוש במסד נתונים NoSQL כמו MongoDB, יכולה לטפל ביעילות בדרישות הנתונים המשתנות של אפליקציית SmallTalk.
- הפרדה לשכבות נפרדות מקדמת תחזוקה של האפליקציה. זה גם מאפשר להתמקד בחזוקה על אבטחה ופרטיות, כאשר המנגנוני הצפנה משולבים בתוך האדריכלות.
- הבחירה בטכנולוגיות כמו Vaadin ו Spring Boot מבטיחה עקביות טכנולוגית ומקלה על האינטגרציה בין רכיבי הלקוח והשרת.

11.3 מודל שרת/לקוח

המערכת **SmallTalk** פועלת לפי **מודל שרת-לקוח (Client-Server Model)** שהוא מודל ארכיטקטוני קלאסי בפיתוח מערכות תקשורת. במודל זה, יש הפרדה ברורה בין שני צדדים:

- **הלקוח – (Client)** האפליקציה שרצה בדפדפן של המשתמש.
- **השרת – (Server)** האחראי על ניהול הנתונים, האבטחה והלוגיקה העסקית.

עקרונות המודל:

- הלקוח שולח בקשות לשרת (למשל: התחברות, שליחת הודעה).
- השרת מקבל את הבקשה, מעבד אותה, ניגש למסד הנתונים (אם צריך), ומחזיר תגובה ללקוח.
- הלקוח מציג את התוצאה למשתמש.

אופן הפעולה במערכת SmallTalk

במערכת משולבים שני סוגים של תקשורת בין הלקוח לשרת:

1. תקשורת באמצעות REST API

- **שימוש עיקרי:** פעולות בסיסיות (התחברות, רישום, שליפת היסטוריית שיחות).
- **פרוטוקול HTTP :**
- **פורמט נתונים JSON :**
- **אופן פעולה:**
 - המשתמש ממלא טופס התחברות → הלקוח שולח בקשת POST לשרת עם הנתונים.
 - השרת בודק את הנתונים מול מסד הנתונים (MongoDB), ואם הכל תקין – מחזיר תגובה עם פרטי המשתמש.
 - הלקוח מציג את המידע על המסך.

2. תקשורת בזמן אמת באמצעות WebSocket

- **שימוש עיקרי:** שליחת הודעות מיידיות, קבלת התראות, סנכרון בזמן אמת.
- **מאפיינים:**
 - פתיחת חיבור קבוע (persistent connection) בין הלקוח לשרת.
 - תקשורת דו-כיוונית – גם השרת יכול לפנות ללקוח בזמן אמת.
- **אופן פעולה:**
 - בעת טעינת האפליקציה, הלקוח פותח חיבור WebSocket מול השרת.
 - כאשר משתמש שולח הודעה → ההודעה מוצפנת (AES) ונשלחת לשרת דרך ה-WebSocket.
 - השרת מקבל את ההודעה, שולח אותה למקבל בזמן אמת – גם כן דרך WebSocket.
 - אצל המקבל: ההודעה מתקבלת, מפוענחת ומוצגת באופן מידי.
- **יתרונות המודל:**
- **הפרדה ברורה בין שכבות** – ממשיך משתמש, לוגיקה עסקית ונתונים.
- **גמישות** – ניתן להחליף/לשדרג את צד הלקוח או השרת בלי לפגוע במערכת כולה.
- **תמיכה בעדכון בזמן אמת** – שילוב REST ו WebSocket-מאפשר גם פעולה רגילה וגם תקשורת חיה.

11.4 תיאור מסד הנתונים

מסד נתונים

מסד הנתונים לפרויקט SmallTalk

סוג מסד נתונים MongoDB .

מאפיינים טכנולוגיים:

- מסד נתונים NoSQL מבוסס ענן.
- אחסון מסמכים גמיש.
- ביצועים גבוהים.
- הרחבה קלה.
- תמיכה מלאה בהצפנה.

אוספים עיקריים

1. **Users Collection** תפקיד: ניהול זהות משתמשים ואבטחת כניסה שדות מרכזיים כוללים

מזהה ייחודי, שם משתמש, כתובת דוא"ל, סיסמא.

```
_id: "ilan@gmail.com"
fullName: "ilan peretz"
password: "1111"
_class: "mu.smalltalk.entitis.User"
```

```
_id: "michaell@gmail.com"
fullName: "michael uliel"
password: "2222"
_class: "mu.smalltalk.entitis.User"
```

```
_id: "danieli@gmail.com"
fullName: "daniel itzhak"
password: "3333"
_class: "mu.smalltalk.entitis.User"
```

2. **Group Collection** תפקיד תיעוד וניהול קשרי תקשורת בין משתמשים שדות עיקריים: מזהה שיחה, רשימת משתתפים,



3. **Messages Collection** תפקיד אחסון והצפנת הודעות בין משתמשים שדות מרכזיים: מזהה הודעה, שיוך לשיחה, פרטי שולח, תוכן מוצפן, חתימה דיגיטלית.

11.5 תהליכים במערכת ההפעלה (Operating System Processes)

מערכת ההפעלה מהווה את התשתית המרכזית להפעלת האפליקציה ומנהלת את כל התהליכים (Processes) שרצים בה. תהליך הוא מופע ריצה של תוכנה – לדוגמה, כל הפעלה של SmallTalk יוצרת תהליך נפרד במערכת, שלו מוקצים משאבים ייעודיים כמו זיכרון, זמן מעבד וגישה למשאבי קלט/פלט ניהול תהליכים

מערכת ההפעלה אחראית על יצירה, תזמון, הקצאת משאבים וסיום של תהליכים. כל תהליך פועל בסביבה מבודדת, ומערכת ההפעלה דואגת להפריד בין התהליכים כדי להבטיח יציבות, אבטחה ומניעת פגיעה הדדית.

במקרה של SmallTalk, כל משתמש שמתחבר לאפליקציה מפעיל תהליך שמטפל בבקשותיו, שולח ומקבל הודעות מוצפנות, ומבצע הצפנה/פענוח של נתונים – כאשר מערכת ההפעלה דואגת שהמשאבים הדרושים יוקצו לכל תהליך לפי הצורך.

מעבר בין מצבי תהליך

תהליך יכול להיות באחד מהמצבים הבאים: ריצה (Running), מוכן (Ready), המתנה (Waiting/Blocked) או סיום (Terminated). מערכת ההפעלה מבצעת תזמון (Scheduling) בין התהליכים, ומחליטה איזה תהליך ירוץ בכל רגע נתון, בהתאם למדיניות (למשל, תעדוף תהליכים קריטיים כמו הצפנה בזמן אמת)

סנכרון ותקשורת בין תהליכים

לעיתים תהליכים צריכים לתקשר, למשל כאשר יש צורך לעדכן משתמשים אחרים בזמן אמת (כמו הודעות ב-WebSocket). מערכת ההפעלה מספקת מגוונים לתקשורת וסנכרון בין תהליכים (Inter-Process Communication – IPC), המאפשרים להעביר מידע בצורה מאובטחת ויעילה בין תהליכים שונים באפליקציה

הגנה והרשאות

מערכת ההפעלה מוודאת שלכל תהליך יש הרשאות מתאימות, ומונעת ממנו לגשת למשאבים או נתונים של תהליכים אחרים ללא הרשאה. כך נשמרת פרטיות המשתמשים והגנה על המידע,

במיוחד בפרויקטים מבוססי סייבר כמו SmallTalk

השפעה על ביצועים ואבטחה

ניהול תהליכים יעיל תורם לביצועים גבוהים של האפליקציה, מאפשר תגובה מהירה של המערכת, ומקטין את הסיכון לפגיעות אבטחה. שילוב נכון של מנגנוני ניהול תהליכים, יחד עם הצפנה מתקדמת, מבטיח חוויית משתמש יציבה ומוגנת.

11.6 תיאור פרוטוקולי תקשורת

תיאור פרוטוקולי תקשורת

- **פרוטוקול WebSocket**

קשר חי ומידי מספק חיבור דו-כיווני רציני בין הלקוח (דפדפן) לבין השרת. זה אומר שאם יש שינויים בנתונים בצד השרת, ניתן לשלוח את העדכונים ללקוח (client) בזמן אמת מבלי להפעיל בקשות חדשות.

- **פרוטוקול HTTP**

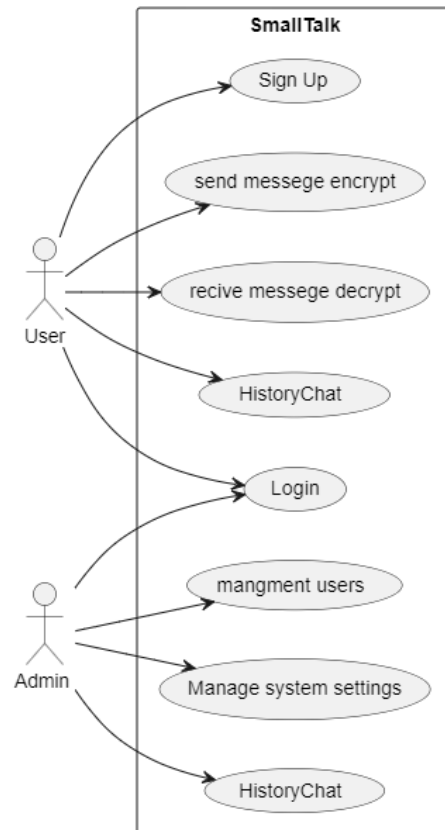
(HTTP (Hypertext Transfer Protocol הוא פרוטוקול המשמש לתקשורת בין שרת האינטרנט לדפדפן האינטרנט של המשתמש. הוא מאפשר לדפדפנים לבקש מידע משרתים ולהציג אותו למשתמשים. HTTP באתר שלי זה ישמש לתקשורת בין הלקוח והשרת.

11.7 REST Api ?

12 ניתוח תרחישים וזרימת המידע

12.1 התרשים המוצג הוא תרשים Use-Case (תרחישי שימוש) עבור מערכת בשם SmallTalk. התרשים מתאר את השחקנים (Actors) במערכת ואת הפעולות המרכזיות (Use Cases) שכל אחד מהם יכול לבצע.

- **User (משתמש רגיל):** משתמש רגיל של המערכת, יכול לבצע פעולות בסיסיות.
- **Admin (מנהל):** בעל הרשאות ניהול, יכול לבצע פעולות מתקדמות וניהוליות.

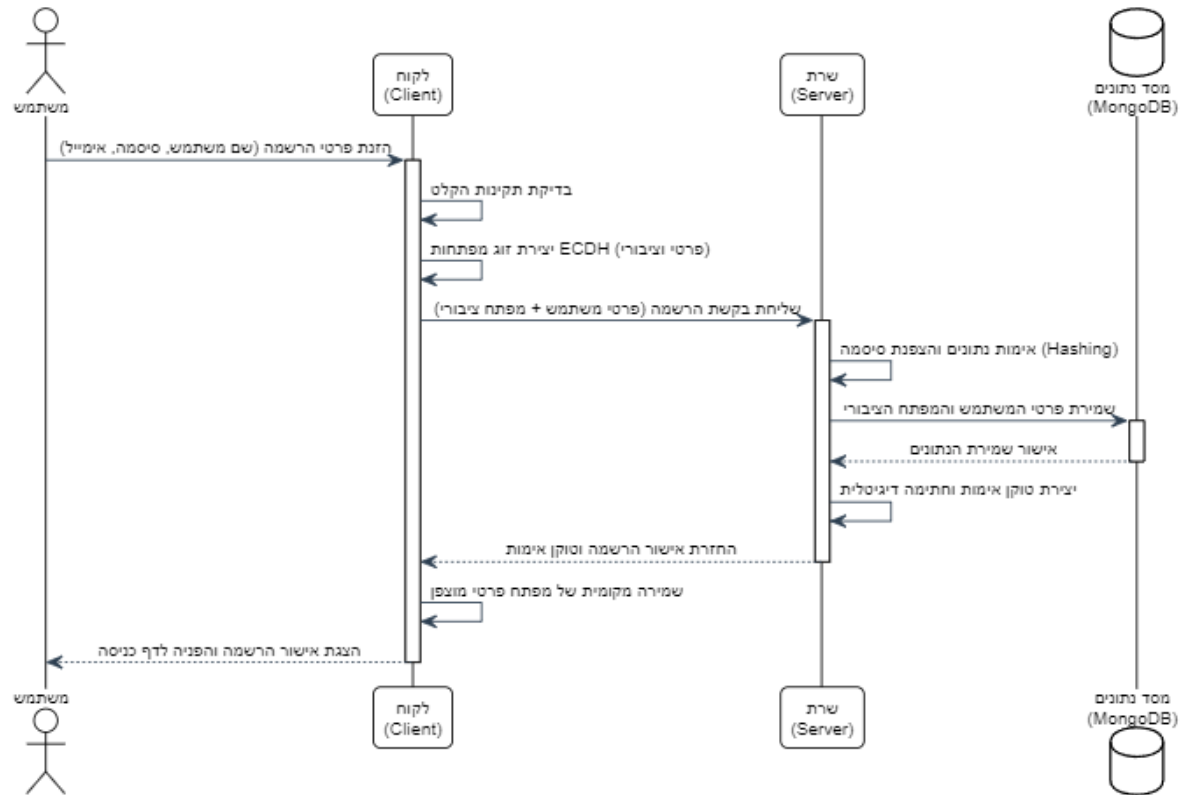


תרחיש שימוש	שחקן מבצע	הסבר
Sign Up	User	הרשמה למערכת. המשתמש יוצר חשבון חדש כדי להתחיל להשתמש בשירות.
send message encrypt	User	שליחת הודעה מוצפנת. המשתמש שולח הודעה שמוצפנת לצורך אבטחת המידע.
recv message decrypt	User	קבלת הודעה ופענוחה. המשתמש מקבל הודעה מוצפנת ומפענח אותה לקריאה.
HistoryChat	User, Admin	צפייה בהיסטוריית הצ'אט. מאפשר למשתמשים ולמנהלים לעיין בהודעות קודמות.
Login	User	כניסה למערכת באמצעות שם משתמש וסיסמה.
mangment users	Admin	ניהול משתמשים. המנהל יכול להוסיף, להסיר או לערוך משתמשים במערכת.
Manage system settings	Admin	ניהול הגדרות מערכת. המנהל יכול לשנות הגדרות כלליות של המערכת.

Sequence Diagram תרשימי רצף

הצגת 3-4 תרשימי Sequence Diagram לתרחישים עיקריים המפרטים את זרימת המידע + הסבר

תהליך הרשמה ויצירת מפתחות קריפטוגרפיים



תרשים זה מתאר את תהליך ההרשמה למערכת ויצירת המפתחות הקריפטוגרפיים הנחוצים לאבטחת התקשורת.

- **משתמש:** האדם המשתמש במערכת
- **לקוח (Client):** אפליקציית הווב בה המשתמש מתפעל את המערכת
- **שרת (Server):** צד השרת של המערכת
- **מסד נתונים (MongoDB):** מסד הנתונים בו נשמרים פרטי המשתמשים

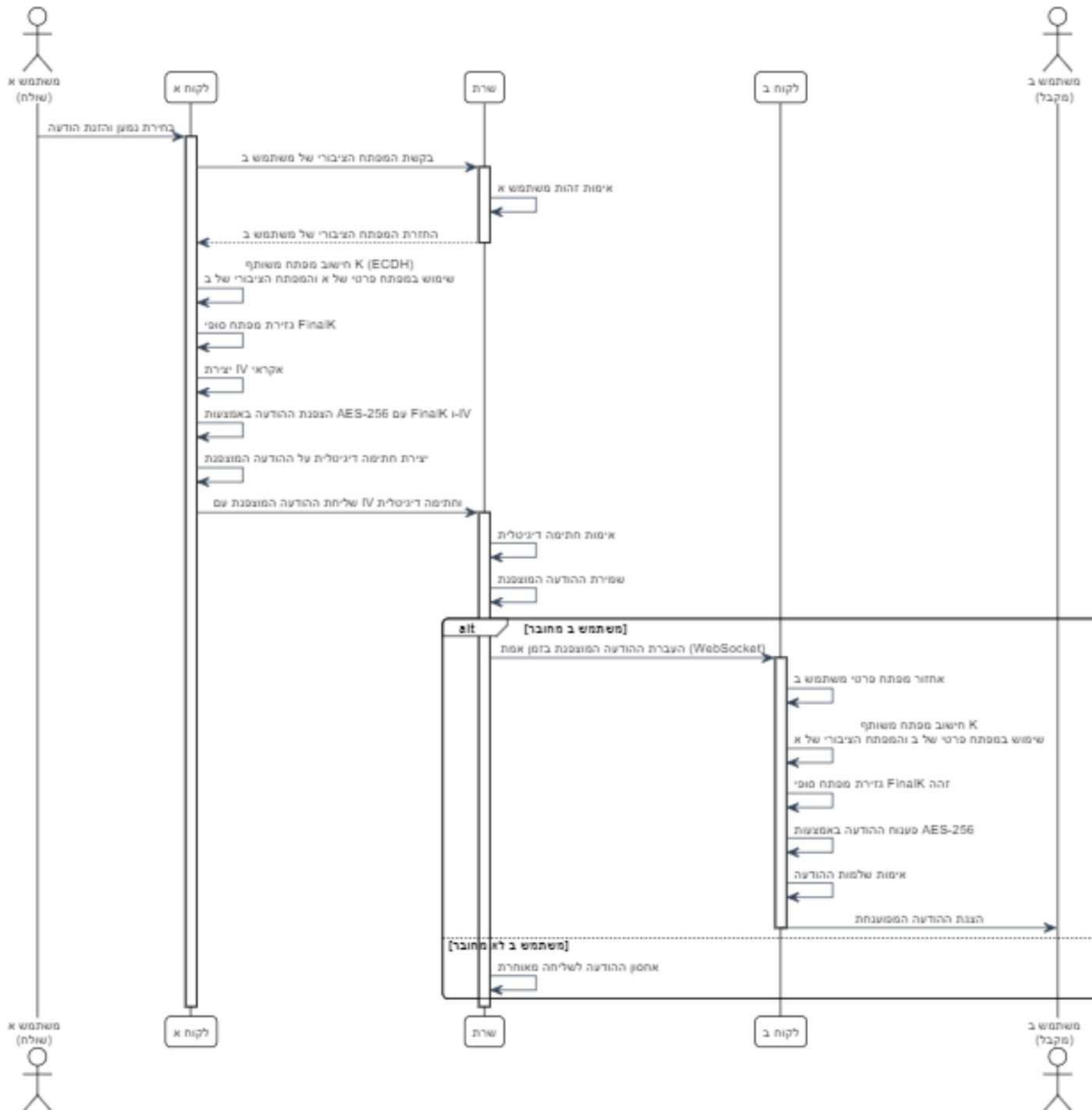
• תהליך:

1. המשתמש מזין את פרטי ההרשמה שלו (שם משתמש, סיסמה, כתובת אימייל).
2. הלקוח מבצע בדיקות תקינות ראשוניות על הקלט כדי לוודא שהוא עומד בדרישות המערכת.
3. **נקודה קריטית לאבטחה:** הלקוח מייצר זוג מפתחות - ECDH מפתח פרטי שנשאר בצד הלקוח, ומפתח ציבורי שנשלח לשרת.
4. הלקוח שולח לשרת את פרטי המשתמש יחד עם המפתח הציבורי.
5. השרת מאמת את הנתונים ומבצע הצפנת Hash לסיסמה (חשוב לציין שהסיסמה עצמה לעולם לא נשמרת במערכת).
6. השרת שומר במסד הנתונים את פרטי המשתמש והמפתח הציבורי.
7. השרת מייצר טוקן אימות עם חתימה דיגיטלית ומחזיר אותו ללקוח.
8. הלקוח שומר באופן מקומי את המפתח הפרטי כשהוא מוצפן, כך שגם אם המכשיר ייפרץ, המפתח יהיה מוגן.
9. המשתמש מקבל אישור על הרשמה מוצלחת ומופנה לדף הכניסה.

• חשיבות התהליך:

תהליך זה מהווה את הבסיס לכל מנגנוני האבטחה במערכת. הוא מייצר את התשתית הקריפטוגרפית הנחוצה להצפנה מקצה לקצה ולכן חייב להיות מאובטח ביותר. היתרון של הגישה הזו היא שהמפתח הפרטי לעולם לא עוזב את המכשיר של המשתמש, מה שמגביר משמעותית את רמת האבטחה.

תהליך החלפת מפתחות ושליחת הודעה מוצפנת

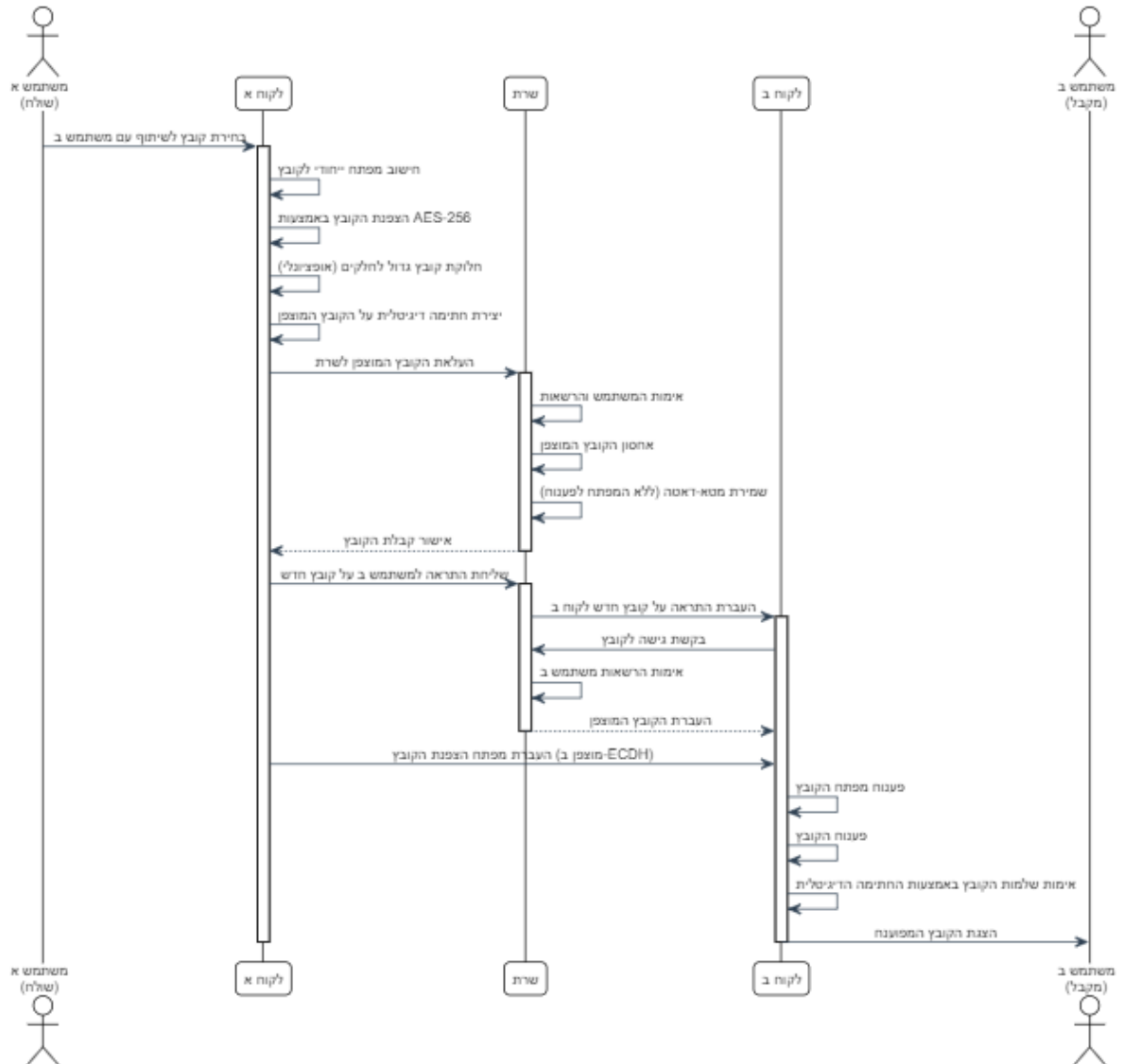


תרשים זה מתאר את הליבה האלגוריתמית של המערכת - הדרך בה מחליפים מפתחות ומצפינים הודעות בין משתמשים.

- **משתמש א (שולח):** המשתמש השולח את ההודעה
- **לקוח א:** אפליקציית הלקוח של המשתמש השולח
- **שרת:** צד השרת של המערכת
- **לקוח ב:** אפליקציית הלקוח של המשתמש המקבל
- **משתמש ב (מקבל):** המשתמש המקבל את ההודעה
- **תהליך:**
 1. משתמש א בוחר נמען (משתמש ב) ומזין הודעה שברצונו לשלוח.
 2. לקוח א מבקש מהשרת את המפתח הציבורי של משתמש ב.
 3. השרת מאמת את זהות משתמש א ומחזיר את המפתח הציבורי של משתמש ב.
 4. **ליבת האלגוריתם: ECDH -** לקוח א מחשב מפתח משותף K באמצעות המפתח הפרטי של משתמש א והמפתח הציבורי של משתמש ב.
 5. הלקוח גוזר מפתח סופי FinalK למשל באמצעות פונקציית SHA-256 .
 6. הלקוח מייצר וקטור אתחול (IV) אקראי למניעת התקפות הצפנה חוזרת.
 7. **הצפנת ההודעה:** הלקוח מצפין את ההודעה באמצעות AES-256 עם המפתח FinalK ו- IV .
 8. הלקוח יוצר חתימה דיגיטלית על ההודעה המוצפנת כדי לאמת את מקור ההודעה.
 9. הלקוח שולח לשרת את ההודעה המוצפנת, ה- IV והחתימה הדיגיטלית.
 10. השרת מאמת את החתימה הדיגיטלית ושומר את ההודעה המוצפנת.
 11. אם משתמש ב מחובר, השרת מעביר את ההודעה המוצפנת בזמן אמת (דרך WebSocket).
 12. לקוח ב מבצע את התהליך ההפוך - חישוב אותו מפתח משותף K באמצעות המפתח הפרטי של משתמש ב והמפתח הציבורי של משתמש א.
 13. לקוח ב גוזר את אותו מפתח סופי FinalK , מפענח את ההודעה, מאמת את שלמותה ומציג אותה למשתמש ב.
 14. אם משתמש ב אינו מחובר, ההודעה נשמרת בשרת לשליחה מאוחרת.
- **חשיבות התהליך:**

זהו התהליך המרכזי במערכת, המדגים את מנגנון ההצפנה מקצה לקצה ($E2EE$). החוזק של המערכת נובע מכך שרק המשתמשים עצמם יכולים לפענח את ההודעות, כי רק להם יש גישה למפתחות הפרטיים. גם השרת עצמו אינו יכול לראות את תוכן ההודעות, מה שמבטיח פרטיות מלאה למשתמשים.

תהליך שיתוף קבצים מוצפנים



תרשים זה מתאר כיצד המערכת מאפשרת שיתוף קבצים מאובטח בין משתמשים.

- **משתמש א (שולח):** המשתמש השולח את הקובץ
- **לקוח א:** אפליקציית הלקוח של המשתמש השולח
- **שרת:** צד השרת של המערכת
- **לקוח ב:** אפליקציית הלקוח של המשתמש המקבל
- **משתמש ב (מקבל):** המשתמש המקבל את הקובץ

• תהליך:

1. משתמש א בוחר קובץ לשיתוף עם משתמש ב.
2. לקוח א מייצר מפתח ייחודי לקובץ.
3. לקוח א מצפין את הקובץ באמצעות AES-256 עם המפתח הייחודי.
4. עבור קבצים גדולים, הלקוח מחלק את הקובץ לחלקים קטנים יותר (chunking) לשיפור ביצועי העלאה.
5. לקוח א יוצר חתימה דיגיטלית על הקובץ המוצפן לוודא את שלמותו.
6. לקוח א מעלה את הקובץ המוצפן לשרת.
7. השרת מאמת את המשתמש והרשאותיו, ומאחסן את הקובץ המוצפן עם מטא-דאטה (ללא המפתח לפענוח).
8. לקוח א שולח התראה למשתמש ב על קובץ חדש (דרך השרת).
9. לקוח ב מקבל התראה על קובץ חדש ומבקש גישה אליו מהשרת.
10. השרת מאמת את הרשאות משתמש ב ומעביר את הקובץ המוצפן ללקוח ב.
11. **נקודה קריטית לאבטחה:** לקוח א מעביר את מפתח הצפנת הקובץ ללקוח ב כשהוא מוצפן באמצעות ECDH דומה לתהליך שליחת הודעות.
12. לקוח ב מפענח את מפתח הקובץ, מפענח את הקובץ עצמו, מאמת את שלמותו באמצעות החתימה הדיגיטלית, ומציג אותו למשתמש ב.

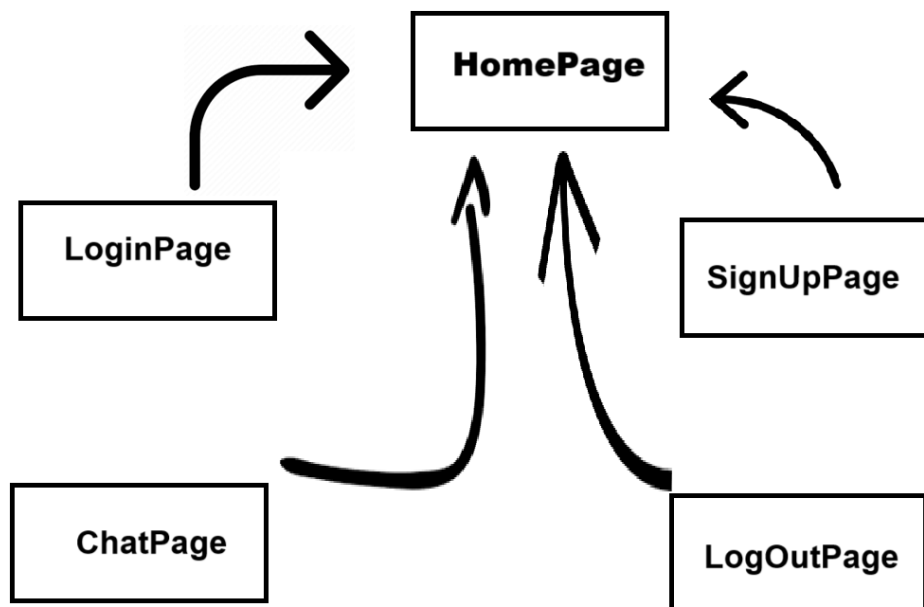
• חשיבות התהליך:

תהליך זה מדגים כיצד המערכת מטפלת בקבצים גדולים תוך שמירה על עקרונות האבטחה. ההפרדה בין הקובץ המוצפן (שנשמר בשרת) לבין המפתח לפענוח (שמועבר בערוץ נפרד) מחזקת את האבטחה. גם אם גורם זדוני יצליח לגשת לקובץ המוצפן בשרת, הוא לא יוכל לפענח אותו ללא המפתח.

13 תיאור מסכים וממשק משתמש

תרשים היררכיית המסכים

תרשים היררכיית מסכים (Screen Flow Diagram)



1. דף הבית (Home)

- מכיל תפריט ניווט עליון עם קישורים: Home, Features, Security, Technology, About, Chat, Login, Logout, Signup.
- כפתור "Learn More" שמוביל לעמוד מידע נוסף.

2. מעבר בין המסכים:

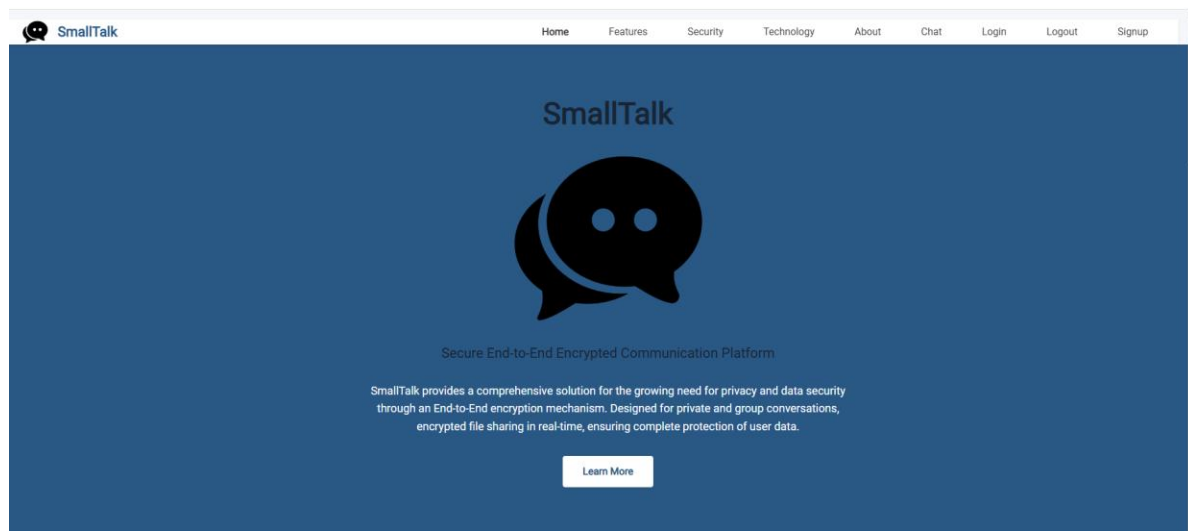
מסך ראשי	מסך יעד	תיאור המעבר
Home	Features	לחיצה על "Features" בתפריט הראשי
Home	Security	לחיצה על "Security" בתפריט הראשי
Home	Technology	לחיצה על "Technology" בתפריט הראשי
Home	About	לחיצה על "About" בתפריט הראשי
Home	Chat	לחיצה על "Chat" בתפריט הראשי
Home	Login	לחיצה על "Login" בתפריט הראשי
Home	Logout	לחיצה על "Logout" בתפריט הראשי
Home	Signup	לחיצה על "Signup" בתפריט הראשי
Home	Learn More	לחיצה על כפתור "Learn More"

3. מסכים נוספים במערכת:

- Features: מסך המציג תכונות המערכת.
- Security: מסך המפרט על מנגנוני האבטחה.
- Technology: מסך עם מידע טכנולוגי.
- About: מסך מידע אודות החברה/המערכת.
- Chat: מסך הציאט (דורש התחברות למערכת).
- Login: מסך התחברות למערכת.
- Logout: מסך/פעולה ליציאה מהמערכת.
- Signup: מסך הרשמה למערכת.

תיאור המסכים

עבור כל מסך (דף): צילום מסך ברור, תיאור תפקידו (למה משמש?), מה הוא מכיל מבחינת ממשק משתמש (GUI)? ועבור כל אלמנט (כדוגמת: כפתור, תיבת טקסט...) יש להסביר את תפקידם. כמו כן, יש לתאר את כל ההודעות/דיאלוגים למשתמש שמוקפצים במסך זה (להציג צילום מסך שלהם ולתאר אותם)



• מסך פתיחה

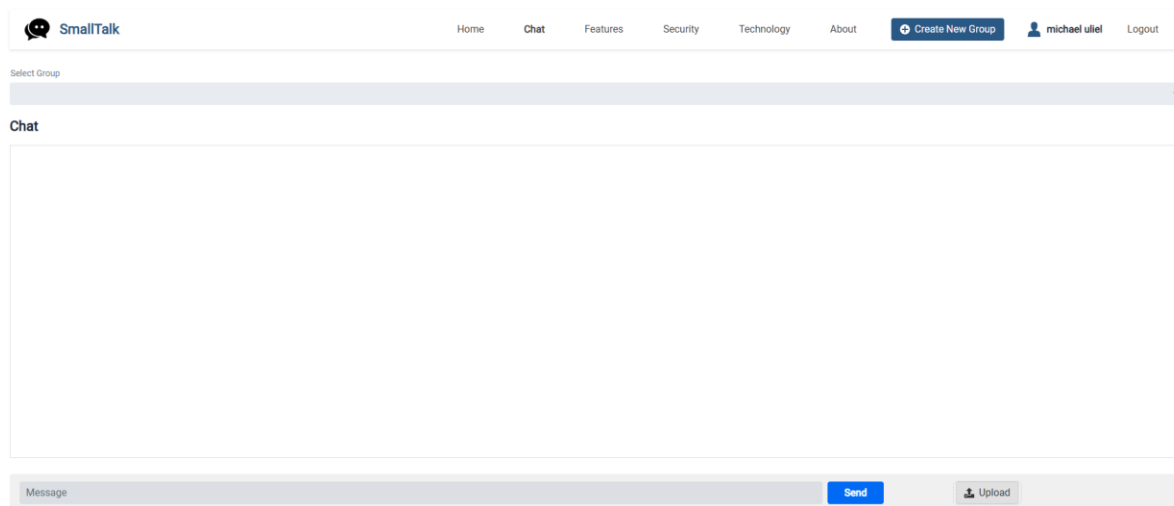
מסך אשר מקבל כל משתמש שנכנס לאתר דרך כתובת url localhost: 8080 בין אם רשום ובין אם לא אך אם אתה לא רשום אתה תקבל Navbar שיציג לך להתחבר ולהרשם ואם לא תרצה להירשם אז לא תוכל להיכנס לצאט

• מסך רישום

• דף Signup אשר אתה יכול דרכו להירשם לאתר – כותב אימייל שם וסיסמה למשתמש

• מסך התחברות

• דף LOGIN אשר דרכו יכול לבצע התחברות לאתר – חייב לרשום אימייל וסיסמה



מסך צאט

- דף chat אשר אתה יכול לדבר ולשוחח ולשלוח לתמונות לקבוצות שאתה שייך אליה

14 תיאור התוכנה

סביבת עבודה

סביבת העבודה בה השתמשתי הוא – Code Studio Visual

השתמשתי ב Spring boot – גרסה 3.4.4

ושפת java גרסה 17.

להלן הפאקגים : ותיאור התוכנה בצורה ברורה יותר איך היא מסודרת ועובדת

Classes כל המחלקות שמשתמשים בהם בפרוייקט נמצאים שם.

Pages שם נמצאים כל הדפים שיכולים להיות מוצגים באתר

Repositories כל המחלקות שמבצעות קשר עם ה DATABASE יש לו Repository

- Services שם נמצאות כל השירותים בשיתוף עם ה Repositories (עם כל ההזרקה של המשתנים)
ונמצאות שם הפעולות העיקריות של הפרוייקט

שפות תכנות

השתמשתי לפרויקט בשפת java גרסה 17

ולחלן הסבר על שפת java

שפת Java היא שפת תכנות עילית, מונחית עצמים, כללית ובטוחה מבחינת ניהול זיכרון. היא פותחה על ידי גיימס גוסלינג בסאן מיקרוסיסטמס ויצאה לראשונה ב-1995. אחת התכונות המרכזיות של Java היא עקרון: Write Once, Run Anywhere (WORA) קוד שנכתב ב Java-ניתן להרצה על כל פלטפורמה התומכת ב, (Java Virtual Machine (JVM)-ללא צורך בהידור מחדש. Java זכתה לפופולריות רבה הודות ליכולת שלה לפעול על מגוון רחב של מערכות הפעלה ומכשירים, בזכות ההידור לבייטקוד והרצה על ה JVM. היא כוללת ניהול זיכרון אוטומטי (garbage collection), מערכת ספריות עשירה, תמיכה בריבוי-תהליכים (multithreading), ומנגנוני אבטחה מובנים היתרונות המרכזיים של: Java:

- עצמאות פלטפורמה: קוד אחד יכול לרוץ על מערכות שונות בזכות ה JVM
- עקרונות מונחי עצמים חזקים: מקלה על ארגון וניהול קוד בפרויקטים גדולים.
- ניהול זיכרון אוטומטי: מפחית דליפות זיכרון ומקל על המפתחים.
- אבטחה מובנית: מנגנונים כמו sandboxing ובדיקת bytecode.
- ספריות וכלים מגוונים: אקוסיסטם עשיר של כלים, מסגרות (frameworks) ותיעוד.
- קהילה פעילה: תמיכה רחבה ועדכונים תכופים.
- תאימות לאחור: גרסאות חדשות שומרות לרוב על תאימות לקוד ישן.

החסרונות הבולטים של: Java:

- ביצועים איטיים יחסית: ההרצה דרך ה JVM-גורמת לעיכוב לעומת שפות כמו C/C++.
 - צריכת זיכרון גבוהה: ניהול הזיכרון האוטומטי וה JVM-דורשים משאבים רבים.
 - קוד ארוך ומסורבל: משימות פשוטות דורשות לעיתים הרבה שורות קוד.
 - מורכבות בריבוי-תהליכים: מימוש וניפוי באגים בתוכניות מרובות תהליכים עשוי להיות מסובך.
 - ממשקי משתמש גרפיים פחות מרשימים: הכלים המובנים ליצירת GUI נחשבים לפחות מתקדמים לעומת טכנולוגיות אחרות.
 - גישה מוגבלת לרמות מערכת נמוכות: פחות מתאימה לפיתוח מערכות הדורשות שליטה ישירה בחומרה.
- Java ממשיכה להתעדכן, והגרסה העדכנית נכון ל-2025 היא Java 24, כאשר גרסאות כמו Java 8, 11 ו-17 עדיין נתמכות לטווח ארוך, השפה מתאימה במיוחד לפיתוח מערכות ארגוניות, אפליקציות אינטרנט, מובייל, מערכות מידע ועוד, אך פחות מתאימה ליישומים הדורשים ביצועים גבוהים במיוחד או שליטה ישירה בחומרה

תיאור המודולים והמחלקות

• עץ המודולים

```
—entitis
    Group.java
    Message.java
    User.java

—Pages
    ChatPage.java
    HomePage.java
    LoginPage.java
    LogoutPage.java
    SignupPage.java

—Repositories
    GroupRepository.java
    MessageRepository.java
    UserRepository.java

—security
    Aes256.java
    Base64FileHandler.java

—Services
    ChatService.java
    DigitalSignatureService.java
    ECDHService.java
    EncryptionService.java
    GlobalMessageBroadcaster.java
    GroupService.java
    MessageService.java
    MongoDBService.java
    UserService.java
```

- תרשים מחלקות Class Diagram
- תיאור מחלקות מפורט

מבנה נתונים בשימוש

קוד התוכנית לפעולות ואלגוריתמים עיקריים וחשובים

לתאר 4-5 פעולות ואלגוריתמים עיקריים וחשובים, וכל אחד : להציג הקוד שלו, תיעוד טענת כניסה ויציאה (פרמטרים וערך מוחזר), מה מבצע, אופן פעולתו בצורה מפורטת, סיבוכיות זמן ריצה...

- פעולה 1

15 מדריך למשתמש

על מנת להריץ את המערכת תצטרך קודם כל שהפרוייקט יהיה אצלך במחשב עליך להוריד למחשב – vscode ואז תצטרך להריץ את השרת . ואז לרשום בדפדפן שיש לך ולרשום localhost:8080 שתיכנס לדף הבית יהיה לך שתי אפשרויות – הרשמה לאתר או כניסה (אם יש משתמש קיים). לאחר שנכנסת למערכת ויש לך משתמש יהיה לך מספר אפשרויות .

יש Navbar שממש כניווט בכל הדפים האלה אפשר לחזור ולהגיע לכל דף

דף בית – עם הסבר על האתר

דף להתנתק – תוכל להתנתק מהמשתמש שלך

דף להתחבר – תוכל להתחבר עם יש לך אימייל וסיסמא אם לא תיצור חשבון

דף להירשם – דף שבו תוכל להירשם למערכת

דף צאט – שם תוכל לדבר ולשוחח ולראות את הקבוצות

16 בדיקות והערכה

למען בדיקת אמינות המערכת בוצעו בפרויקט זה מספר רחב של בדיקות שנועדו לוודא שהמערכת פועלת כהלכה ולא בוצעו בה שגיאות לא רצויות בצד הלקוח ובצד השרת. להלן חלק מהבדיקות אותם ביצעתי בפרויקט

התחברות למערכת ובדיקת כל ההרשאות של הפרויקט. עברתי על כל הבקשות המתבצעות בצד הלקוח הדורשים הרשאות משתמש ובדקתי האם השרת פועל לפי ההרשאות של כל משתמש וחוסם דברים שלמשתמש לא צריך להיות גישה אליהם.

העברת בקשות התחברות שונות הכוללות בקשות HTTP והתחברות מכמה משתמשים

Tomcat (בעל השרת) בדיקת שליחת בקשות לשרת ממספר משתמשים בו זמנית לבדיקה האם השרת יכול לטפל בכל הבקשות בו זמני.

17 מסקנות

במהלך העבודה על הפרויקט, התמודדתי לראשונה עם אתגרי ניהול זמן משמעותיים. נדרשתי לעמוד בלוחות זמנים צפופים, לשלב בין דרישות הפרויקט לבין משימות שוטפות בלימודים, ולעבוד במקביל על כתיבת הדוח, הפיתוח והמחקר. ההתמודדות הזו חידדה אצלי את החשיבות של תכנון נכון, חלוקת משימות, ועבודה מסודרת לפי סדרי עדיפויות.

במהלך הפרויקט נחשפתי לטכנולוגיות חדשות, במיוחד לסביבת Spring Boot ולעבודה מול מוד נתונים (DB). למדתי כיצד לממש פעולות CRUD (יצירה, קריאה, עדכון ומחיקה) מול השרת, ולהבין לעומק כיצד כל התהליך הזה פועל מאחורי הקלעים. תהליך הלמידה כלל חקירה עצמאית, קריאה ממקורות שונים, והשוואה בין גישות טכנולוגיות כדי לבחור את הפתרון המתאים ביותר לפרויקט ולצרכים שלי.

בנוסף, למדתי להיעזר בחברי לכיתה בעת הצורך, וגם לתרום מהידע שלי לאחרים כאשר התאפשר לי. הבנתי שלמידה משותפת ושיתוף פעולה בצוות לא רק מקלים על ההתמודדות עם תקלות ובעיות, אלא גם מעשירים את הידע והניסיון של כל אחד מאיתנו.

העמקתי בחשיבות המחקר העצמאי – גיליתי שככל שמעמיקים בנושא מסוים, כך אפשר להגיע להבנה רחבה יותר, ולשלב בין מקורות מידע שונים כדי ליצור פתרון איכותי ומותאם אישית. למדתי להציג ולהסביר את הפתרונות שמצאתי, לפרט את שלבי העבודה, ולהנגיש את המערכת שבניתי כך שתהיה ברורה ומובנת למשתמשים.

לבסוף, ההתמודדות עם תקלות טכניות, הן שלי והן של אחרים, חיזקה אצלי את החשיבות של עבודת צוות, סבלנות, והיכולת לחלוק ידע וניסיון – תכונות חיוניות להצלחה בכל פרויקט טכנולוגי.

18 פיתוחים עתידיים

1. הפיצ'ר סטטוס אישי דינמי (כמו בוואטסאפ):
 - יאפשר למשתמשים להעלות עדכון אישי זמני (טקסט/תמונה/סרטון)
 - העדכון יוצג לחברים/קשרים במערכת למשך 24 שעות
 - נותן הצצה מהירה למה שקורה לאנשים בזמן אמת.
2. שיחה קולית/וידאו הטמעת אפשרות לתקשורת סינכרונית דרך שיחות קוליות או וידאו תשדרג משמעותית את יכולות התקשורת במערכת, ותאפשר אינטראקציה אנושית יותר וישירה
3. יכולות תרגום מקוונות בזמן אמת הוספת מנגנון תרגום אוטומטי מידי שיאפשר תקשורת בין שפות ללא מגבלות. המערכת תתרגם הודעות באופן אוטומטי בזמן אמת, מה שיפרוץ מחסומי שפה ויאפשר תקשורת גלובלית יותר.

19 בבליוגרפיה

1. Robby K. H, Antonius I.S, Dyah N, Widyastuti W, "Securing RFID in IoT Networks with Lightweight AES and ECDH Cryptography Approach", August 2024
<https://www.researchgate.net/publication/383236735>
2. Limin Zhang, Li Wang, "A Hybrid Encryption Approach for Efficient and Secure Data Transmission in IoT Devices", June 2024
<https://jeas.springeropen.com/articles/10.1186/s44147-024-00459-x>
3. הסבר על אלגוריתם AES 256
<https://www.wallarm.com/what/what-is-aes-advanced-encryption-standard>